

MAMBA-Frost

Specification for a Key Encapsulation Mechanism from Learning with Quantization

Shanxiang Lyu^{1*} Ke Ma¹ Junzuo Lai^{1*}
Zheng Qin² Ming Li³

¹ College of Cyber Security, Jinan University, Guangzhou, China

² Shenzhen Institute of Beihang University, Shenzhen, China

³ PEI DINGYI Institute of Cryptology, Guangzhou University, Guangzhou, China

* Correspondence: lsx07@jnu.edu.cn, laijunzuo@gmail.com

JUNE 30, 2026

Abstract

The *Learning With Errors* (LWE) problem provides a conservative and well-established security foundation for lattice-based cryptography, while *Learning With Rounding* (LWR) improves bandwidth efficiency through deterministic rounding. However, the rounding noise in LWR is inherently correlated with the hidden linear term, preventing tight and sample-preserving reductions to standard LWE for polynomial moduli.

In this work, we revisit the *Learning With Quantization* (LWQ) problem and show that dithered quantization restores the additive-noise structure that deterministic rounding destroys. With public dithering, the induced quantization error is statistically independent of the hidden input, and the resulting LWQ transcript admits a tight, sample-preserving correspondence with LWE. Building on this foundation, we propose **Frost**, a plain LWQ-based key encapsulation mechanism that combines unstructured LWE security with quantization-based compression. By using LWQ as the transcript structure, **Frost** derives its effective error from public dithered quantization rather than from an explicit Gaussian sampler. The construction uses power-of-two arithmetic throughout and employs an E_8 -coded message embedding to improve the correctness margin without changing the underlying LWQ hardness argument.

We instantiate **Frost** across five security profiles from 128 to 512 bits and provide both a portable reference implementation and an AVX2 implementation in the submitted API_PKC package. Its power-of-two design realizes the core arithmetic with shifts and masks, keeping the implementation simple and amenable to constant-time engineering. The submitted package is evaluated through the official API_PKC interface and auxiliary-function path, while a native AES128-backend benchmark is reported separately to measure the underlying arithmetic implementation. In the native benchmark with E_8 coding, **Frost-128** has a 5152-byte public key and a 5192-byte ciphertext, and the measured AVX2 cycle counts are 401, 569, and 666 kCycles for key generation, encapsulation, and decapsulation.

Contents

1	Introduction	3
1.1	Design Rationale	4
2	General Algorithm Specification	5
2.1	Notation	5
2.2	Frost PKE	6
2.3	Frost KEM	8
3	Parameter Settings	9
3.1	Default Parameter Sets	9
3.2	Compact-Ciphertext Variants	10
4	Known Answer Tests	11
5	Efficiency Evaluation	12
5.1	Benchmarking Platforms	12
5.2	x86-64 API_PKC Self-Evaluation Results	12
5.3	Cortex-M4 Efficiency Results	13
6	Correctness and Security Analysis	13
6.1	Security Definitions and Claims	14
6.2	Correctness of the PKE Scheme	15
6.3	Theoretical Security	16
6.4	Concrete Security	17
7	Comparison with Similar Lattice KEMs	17
7.1	Comparison with FrodoKEM	18
7.2	Comparison with Scloud ⁺	19
7.3	Other LWE/LWR-Based KEMs	19
8	Advantages and Limitations	20
8.1	Advantages	20
8.2	Limitations	20
8.3	Side-Channel Considerations	20
9	Implementation Specification	21
9.1	External API and Implementation Boundary	21
9.2	Integer Representation	21
9.3	Serialized Objects	22
9.4	Bit Packing and Parsing	22
9.5	Seed Expansion and Matrix Generation	22
9.6	Secret Sampling	23
9.7	Dithered Quantization and Compensation	23
9.8	Message Encoding and Decoding	24
9.9	Validation Requirements	25
A	Mathematical Foundations	26
B	Correctness and Security Details	27
B.1	Lattice Code and DFR Analysis	27
B.2	Proof of the LWQ-to-LWE Correspondence	28
B.3	Proof of IND-CPA Security	28

B.4 Proof of IND-CCA Security	29
C Native x86 Implementation Benchmarks	30

1 Introduction

Frost (formally designated as **MAMBA-Frost**) is a lattice-based key encapsulation mechanism (KEM) constructed from the Learning with Quantization (LWQ) problem over unstructured integer lattices [LLL24a]. The construction first defines an IND-CPA secure public-key encryption scheme, denoted as **Frost.PKE**, and subsequently applies a Fujisaki–Okamoto transformation to derive the IND-CCA secure key encapsulation mechanism **Frost.KEM** [FO99, HHK17, JZC⁺18]. This specification provides a self-contained exposition of the underlying algorithms, concrete parameter sets, byte encodings, implementation interfaces, correctness bounds, and security rationales.

Frost operates entirely over unstructured matrices over $\mathbb{Z}_q$, leveraging public dithered quantization and power-of-two arithmetic. Public keys and ciphertext components are transmitted in a compressed, split representation. Upon applying public dither compensation, the resulting transcript admits an additive normal form whose effective errors are fully induced by the quantizer and remain statistically independent of the hidden quantization input [Zam14, LLL24a]. This independence constitutes the core structural advantage of **Frost**: the cryptographic security reduction anchors tightly to the standard, unstructured Learning With Errors (LWE) problem [Reg05, BLP⁺13], while the quantization mechanism inherently delivers compact communication bandwidth.

All modular reductions, compression, reconstruction, packing, and unpacking operations are strictly specified over fixed-width integers. The scheme deliberately eschews polynomial rings, module structures, Number Theoretic Transform (NTT) arithmetic, Gaussian sampling, and rejection sampling. This high-level algorithmic framework remains invariant across all parameter profiles. A concrete profile is determined by fixing the matrix dimensions, the primary modulus, compression bit-widths, secret distributions, message-encoding parameters, and the shared-secret length.

This document specifies five default parameter sets: **Frost-128**, **Frost-192**, **Frost-256**, **Frost-384**, and **Frost-512**. These profiles target escalating classical security margins and provide corresponding quantum security estimates. For the two highest security levels, this specification additionally defines compact-ciphertext variants (denoted with the suffix **-CC**) that transpose the two message dimensions while preserving the underlying correctness and security reductions up to algebraic transposition. Table 1 summarizes the high-level structural distinctions between **Frost** and other representative KEM paradigms. A comprehensive executive summary detailing the concrete bandwidth overheads, decryption failure rates, and concrete bit-security margins across all proposed **Frost** profiles (including the compact-ciphertext variants) is provided in Table 2.

Table 1: High-level architectural comparison of **Frost** with representative unstructured KEMs.

Feature	FrodoKEM	SCloud ⁺	Frost (Ours)
Underlying Problem	Plain LWE	Plain LWE	Plain LWQ
Modulus Arithmetic	Power-of-two / Prime	Power-of-two (2^{12})	Power-of-two ($2^{15}/2^{16}$)
Secret Distribution	Discrete Gaussian	Fixed-weight Ternary	Centered Binomial
Error / Noise Source	Explicit Sampling	Explicit Sampling	Public Dithered Quantization
Error Correction Layer	Scalar Rounding	BW_{32} Lattice Code	E_8 Lattice Code
Bandwidth Compression	Rounding	Rounding	Inherent from Quantization

Table 2: Executive summary of *Frost* profiles: bandwidth, decryption-failure upper bounds, and concrete security margins.

Scheme Profile	pk (B)	ct (B)	sk _{KEM} (B)	$\log_2 \delta_{\text{dec}}$	MATZOV	Core-C	Core-Q
Frost-128	5152	5192	6736	-169.44	131.85	107.75	98.58
Frost-192	9712	9760	11528	-199.84	197.65	175.27	161.13
Frost-256	16776	15552	19416	-266.61	257.08	236.36	216.55
Frost-384	25096	37736	29032	-440.52	393.79	376.83	345.16
Frost-CC-384	37628	25204	43492	-440.52	393.79	376.83	345.16
Frost-512	36432	72944	41728	-581.12	512.97	498.68	456.47
Frost-CC-512	72832	36544	83328	-581.12	512.97	498.68	456.47

1.1 Design Rationale

This section delineates the foundational design principles of *Frost*. The primary cryptographic primitive underlying *Frost* is LWQ, which reduces directly to the classical LWE problem. The overriding design objective is to achieve bandwidth-efficient representations without introducing structured algebraic lattices, thereby preserving a tight and transparent reduction to standard, unstructured lattice assumptions.

1.1.1 LWQ as the Security Primitive

The central abstraction of *Frost* is the LWQ framework, wherein ciphertext compression and noise generation are jointly realized through a unified quantization process, rather than via explicit, independent additive error sampling [LLL24a]. Under this paradigm, LWQ is treated not as a mere optimization layer superimposed on LWE, but as a standalone cryptographic primitive whose concrete security is bound to LWE via a tight reduction. Randomness is injected into the system via a public dither prior to quantization, and the identical dither is subsequently subtracted during reconstruction. Consequently, noise is not an independently injected variable; rather, it is intrinsically induced as the residual of the quantization mechanism. Under a uniformly distributed public dither, this quantization error becomes statistically independent of the input.

Although LWQ is formalized via a representation-theoretic mechanism rather than an explicit additive noise model, its provable security is anchored in a reduction to standard LWE. The public dithered quantization operator induces a bounded-support error distribution that remains independent of the secret-dependent signal. This independence allows the resulting transcript to be cleanly reinterpreted as a standard LWE instance with an explicitly characterized noise distribution. In this reduction, LWE serves as the target distribution, while LWQ acts as the randomized representation mechanism that yields an LWE-compatible statistical profile without relying on algebraic ring structures. This decoupling unifies compression and noise generation into a single algorithmic step while maintaining a sharp, non-asymptotic reduction to classical lattice assumptions.

1.1.2 Power-of-Two Arithmetic and Implementation Efficiency

To maximize hardware and software efficiency, *Frost* mandates power-of-two moduli and quantization granularities across all coefficient-level operations. This architectural choice ensures that modular reduction, bit-packing, and reconstruction can be executed exclusively via bitwise shifts, masks, additions, and fixed-point rounding. By eliminating general modular inversion and rejection sampling, *Frost* inherently simplifies constant-time implementations and minimizes side-channel leakage surfaces. Notably, both compression and decompression routines are entirely deterministic, requiring no data-dependent branching or variable-time arithmetic. While power-of-two moduli are notoriously incompatible with NTT-friendly ring structures, *Frost* circumvents this limitation by operating over standard LWE-style linear algebra rather than structured ring arithmetic. The absence

of polynomial multiplication allows all operations to be performed as native integer matrix-vector products, perfectly aligning with the LWQ representation model.

1.1.3 Lattice Coding for Error Correction

To enhance decryption reliability without perturbing the underlying LWQ security distribution, **Frost** integrates an E_8 root-lattice coding layer for message encoding. The E_8 lattice—the unique even unimodular lattice in eight dimensions—exhibits exceptional packing density and rapid decoding properties, rendering it ideal for high-reliability error correction with minimal computational overhead. Compared to higher-dimensional counterparts, such as the 24-dimensional Leech lattice or 32-dimensional Barnes–Wall (BW_{32}) constructions, E_8 offers an optimal trade-off between decoding complexity and coding gain. Its nearest-lattice decoding can be implemented efficiently via simple sorting and rounding algorithms while substantially widening the noise-tolerance margin of the cryptosystem.

In **Frost**, the lattice code is strictly decoupled from the LWQ sampling layer; it exerts no influence on the security reduction and operates solely at the plaintext-ciphertext interface. For default parameter sets, E_8 coding is applied column-wise. For compact-ciphertext, high-security variants, the identical coding procedure is applied row-wise. This structural symmetry is preserved under the parameter constraints $n = m$, $\eta_s = \eta_r$, and $t_{\text{pk}} = t_u$, ensuring that the correctness and decryption failure analyses remain invariant up to a matrix transposition.

2 General Algorithm Specification

This section gives the top-level algorithm specification for **Frost**. It fixes the notation used by the algorithms, defines the IND-CPA public-key encryption scheme **Frost.PKE**, and defines the IND-CCA key encapsulation mechanism **Frost.KEM**. Bit-exact encodings, serialized formats, supporting functions, and implementation constants are specified in Section 9.

2.1 Notation

For an integer $q \geq 1$, let $\mathbb{Z}_q$ denote the integers modulo q , represented by values in $\{0, \dots, q-1\}$. For $x \in \mathbb{Z}$, $x \bmod q$ denotes its canonical representative in $\{0, \dots, q-1\}$, and $x^\pm \bmod q$ denotes its centered representative in $\{-\lfloor q/2 \rfloor, \dots, \lfloor q/2 \rfloor - 1\}$. All additions and multiplications over $\mathbb{Z}_q$ are reduced modulo q . For $x \in \mathbb{R}$, write $\lfloor x \rfloor = \lfloor x + 1/2 \rfloor$ for nearest-integer rounding with ties rounded upward. All scalar operations extend entrywise to vectors and matrices.

Matrices are written in bold uppercase letters and vectors in bold lowercase letters. Transposition is denoted by $^\top$. For a finite set S , $\mathcal{U}(S)$ denotes the uniform distribution over S . For a distribution χ , $x \leftarrow \chi$ denotes sampling from χ , and $\mathbf{X} \leftarrow \chi^{a \times b}$ denotes entrywise independent sampling of an $a \times b$ matrix. We write $\Pr[\cdot]$ and $\mathbb{E}[\cdot]$ for probability and expectation.

For $\eta \geq 1$, the centered binomial distribution $\mathcal{B}_\eta$ is the distribution of $X = \sum_{i=1}^\eta (a_i - b_i)$, where $a_i, b_i \leftarrow \mathcal{U}(\{0, 1\})$ are independent. Thus $\text{supp}(\mathcal{B}_\eta) = \{-\eta, \dots, \eta\}$, $\mathbb{E}[X] = 0$, and $\text{Var}(X) = \eta/2$.

An N -dimensional lattice $\Lambda \subset \mathbb{R}^N$ is a discrete additive subgroup. For a full-rank basis matrix $\mathbf{B}$, write $\mathcal{L}(\mathbf{B}) = \{\mathbf{B}\mathbf{z} : \mathbf{z} \in \mathbb{Z}^N\}$. The nearest-neighbor quantizer is $\mathbf{Q}_\Lambda(\mathbf{x}) = \arg \min_{\boldsymbol{\lambda} \in \Lambda} \|\mathbf{x} - \boldsymbol{\lambda}\|$, with ties broken by a fixed translation-consistent rule. Given a public dither $\mathbf{d}$, write $\mathbf{Q}_{\Lambda, \mathbf{d}}(\mathbf{x}) = \mathbf{Q}_\Lambda(\mathbf{x} + \mathbf{d})$, and define the compensated quantization error as $\mathbf{e}_\mathbf{Q} = \mathbf{Q}_\Lambda(\mathbf{x} + \mathbf{d}) - (\mathbf{x} + \mathbf{d})$.

Each **Frost** profile fixes a security level λ , dimensions m, n, ℓ_r, ℓ_s , modulus q , compression widths t_{pk}, t_u, t_v , centered-binomial parameters η_s, η_r , FO plaintext length κ_m , and shared-secret length κ_{ss} . For all profiles considered in this document, $\kappa_m = \kappa_{\text{ss}} = \lambda$ and $b_{\text{msg}} \ell_r \ell_s = \kappa_m$. The default parameter sets use $\ell_r = 8$ and apply the message codec column-wise. The compact-ciphertext variants for the high-security profiles exchange ℓ_r and ℓ_s , so that $\ell_s = 8$ and the same 8-dimensional message layer is applied row-wise.

The FO plaintext space is $\mathcal{M}_{\text{FO}} = \{0, 1\}^{\kappa_m}$. Messages are encoded and decoded by **Encode** : $\mathcal{M}_{\text{FO}} \rightarrow \mathbb{Z}_q^{\ell_r \times \ell_s}$, **Decode** : $\mathbb{Z}_q^{\ell_r \times \ell_s} \rightarrow \mathcal{M}_{\text{FO}}$.

For $x \in \{\text{pk}, \text{u}, \text{v}\}$, set $p_x = 2^{t_x}$ and $\Delta_x = q/p_x$. The product quantization lattices are $\Lambda_{\text{pk}} = \Delta_{\text{pk}}\mathbb{Z}^{m\ell_r}$, $\Lambda_{\text{u}} = \Delta_{\text{u}}\mathbb{Z}^{n\ell_s}$, $\Lambda_{\text{v}} = \Delta_{\text{v}}\mathbb{Z}^{\ell_r\ell_s}$. The corresponding dither matrices are $\mathbf{D}_{\text{pk}} \in \{0, \dots, \Delta_{\text{pk}} - 1\}^{m \times \ell_r}$, $\mathbf{D}_{\text{u}} \in \{0, \dots, \Delta_{\text{u}} - 1\}^{n \times \ell_s}$, $\mathbf{D}_{\text{v}} \in \{0, \dots, \Delta_{\text{v}} - 1\}^{\ell_r \times \ell_s}$.

The public matrix is $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$. The long-term secret matrix is $\mathbf{S} \in \mathbb{Z}^{n \times \ell_r}$, and the compact public-key matrix is $\mathbf{B} \in \mathbb{Z}_{p_{\text{pk}}}^{m \times \ell_r}$. The ephemeral secret matrix is $\mathbf{R} \in \mathbb{Z}^{m \times \ell_s}$. The compact ciphertext matrices are $\mathbf{U} \in \mathbb{Z}_{p_{\text{u}}}^{n \times \ell_s}$ and $\mathbf{V} \in \mathbb{Z}_{p_{\text{v}}}^{\ell_r \times \ell_s}$.

The public key, PKE secret key, KEM secret key, ciphertext, public-key hash, message, and shared secret are denoted by pk , sk_{PKE} , sk_{KEM} , ct , h_{pk} , M , and ss , respectively. The fallback string used by implicit rejection is $z \in \{0, 1\}^{\kappa_{\text{ss}}}$. The intermediate key material is denoted by $K' \in \{0, 1\}^{\kappa_{\text{ss}}}$. Thus both z and K' have the same bit length as the final shared secret for the selected profile.

The seed ρ determines $(\mathbf{A}, \mathbf{D}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$. The encryption coins r_{Enc} are parsed as (σ, μ) , where σ determines $\mathbf{R} \leftarrow \text{SampleCBD}_{\eta_r}(\sigma, m, \ell_s)$, and μ determines $(\mathbf{D}_{\text{u}}, \mathbf{D}_{\text{v}}) \leftarrow \text{GenDither}(\mu)$. The interfaces $\mathcal{G}_{\text{FO}}, \mathcal{H}_{\text{pk}}, \mathcal{H}_{\text{ct}}, \mathcal{H}_K$ are domain separated and are used for FO coin and key derivation, public-key hashing, ciphertext hashing, and final shared-secret derivation.

The formal LWE, LWQ, Split-LWQ, lattice-code, and security-experiment definitions are given in Appendix A.

2.2 Frost PKE

Frost.PKE consists of three algorithms (Frost.PKE.KeyGen, Frost.PKE.Enc, Frost.PKE.Dec). The public key contains a seed for regenerating the public matrix and the public-key dither, together with a compact public-key matrix. The secret key contains the small secret matrix. Encryption is deterministic once $(\text{pk}, M, r_{\text{Enc}})$ is fixed.

Algorithm 1 Frost.PKE.KeyGen()

- 1: $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$.
 - 2: $(\mathbf{A}, \mathbf{D}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$.
 - 3: $\mathbf{S} \leftarrow \text{SampleCBD}_{\eta_s}(n, \ell_r)$.
 - 4: $\mathbf{B} \leftarrow \mathbf{Q}_{\Lambda_{\text{pk}}, \mathbf{D}_{\text{pk}}}(\mathbf{AS})/\Delta_{\text{pk}}$.
 - 5: $\text{pk} \leftarrow (\rho, \mathbf{B})$.
 - 6: $\text{sk}_{\text{PKE}} \leftarrow \mathbf{S}$.
 - 7: **return** $(\text{pk}, \text{sk}_{\text{PKE}})$.
-

Algorithm 2 Frost.PKE.Enc(pk, M, r_{Enc})

Require: Public key $\text{pk} = (\rho, \mathbf{B})$, message $M \in \mathcal{M}_{\text{FO}}$, and coins r_{Enc} .

- 1: Parse r_{Enc} as (σ, μ) .
 - 2: $(\mathbf{A}, \mathbf{D}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$.
 - 3: $\mathbf{R} \leftarrow \text{SampleCBD}_{\eta_r}(\sigma, m, \ell_s)$.
 - 4: $(\mathbf{D}_{\text{u}}, \mathbf{D}_{\text{v}}) \leftarrow \text{GenDither}(\mu)$.
 - 5: $\hat{\mathbf{B}} \leftarrow \Delta_{\text{pk}}\mathbf{B} - \mathbf{D}_{\text{pk}} \pmod{q}$.
 - 6: $\mathbf{U} \leftarrow \mathbf{Q}_{\Lambda_{\text{u}}, \mathbf{D}_{\text{u}}}(\mathbf{A}^T \mathbf{R})/\Delta_{\text{u}}$.
 - 7: $\mathbf{V} \leftarrow \mathbf{Q}_{\Lambda_{\text{v}}, \mathbf{D}_{\text{v}}}(\hat{\mathbf{B}}^T \mathbf{R} + \text{Encode}(M))/\Delta_{\text{v}}$.
 - 8: $\text{ct} \leftarrow (\mu, \mathbf{U}, \mathbf{V})$.
 - 9: **return** ct .
-

Algorithm 3 Frost.PKE.Dec($\text{sk}_{\text{PKE}}, \text{ct}$)

Require: Secret key $\text{sk}_{\text{PKE}} = \mathbf{S}$ and ciphertext $\text{ct} = (\mu, \mathbf{U}, \mathbf{V})$.

- 1: $(\mathbf{D}_u, \mathbf{D}_v) \leftarrow \text{GenDither}(\mu)$.
 - 2: $\hat{\mathbf{U}} \leftarrow \Delta_u \mathbf{U} - \mathbf{D}_u \pmod{q}$.
 - 3: $\hat{\mathbf{V}} \leftarrow \Delta_v \mathbf{V} - \mathbf{D}_v \pmod{q}$.
 - 4: $\mathbf{W} \leftarrow \hat{\mathbf{V}} - \mathbf{S}^\top \hat{\mathbf{U}} \pmod{q}$.
 - 5: $M' \leftarrow \text{Decode}(\mathbf{W})$.
 - 6: **return** M' .
-

The public key is $\text{pk} = (\rho, \mathbf{B})$, where ρ determines $\mathbf{A}$ and $\mathbf{D}_{\text{pk}}$. The ciphertext is the mathematical tuple $\text{ct} = (\mu, \mathbf{U}, \mathbf{V})$, where μ determines the ciphertext-side dithers. The seed μ is transmitted explicitly so that decryption and FO re-encryption regenerate the same public dithers. The byte serialization order is fixed separately in Section 9.

The message codec used by **Encode** and **Decode** is specified by the following two procedures. Let $\alpha_{E_8} = q/2^{b_{\text{msg}}}$, and let $\mathcal{C}_{E_8} = \alpha_{E_8} E_8 \cap [0, q)^8$. The profile fixes the coordinate bit-width vector

$$\mathbf{b} = (b_{\text{msg}} - 1, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}} + 1),$$

whose entries sum to $8b_{\text{msg}}$. The public maps $\text{Label}_{E_8, \mathbf{b}}$ and $\text{Delabel}_{E_8, \mathbf{b}}$ are inverse bijections between $\prod_{h=0}^7 \{0, \dots, 2^{b_h} - 1\}$ and $\mathcal{C}_{E_8}$. The concrete bit order and the implementation of these maps are fixed in Section 9.

Algorithm 4 Encode(M)

Require: Message $M \in \mathcal{M}_{\text{FO}}$.

- 1: Initialize $\mathbf{Y} \leftarrow \mathbf{0} \in \mathbb{Z}_q^{\ell_r \times \ell_s}$.
 - 2: Parse M into $N_{\text{blk}} = \ell_r \ell_s / 8$ consecutive blocks $M_0, \dots, M_{N_{\text{blk}}-1}$, each of length $8b_{\text{msg}}$ bits.
 - 3: **for** $j = 0, \dots, N_{\text{blk}} - 1$ **do**
 - 4: Parse M_j into integers $(a_{j,0}, \dots, a_{j,7})$, where $0 \leq a_{j,h} < 2^{b_h}$.
 - 5: $\mathbf{c}_j \leftarrow \text{Label}_{E_8, \mathbf{b}}(a_{j,0}, \dots, a_{j,7}) \in \mathcal{C}_{E_8}$.
 - 6: **if** $\ell_r = 8$ **then**
 - 7: Set $\mathbf{Y}_{*,j} \leftarrow \mathbf{c}_j$.
 - 8: **else**
 - 9: Set $\mathbf{Y}_{j,*} \leftarrow \mathbf{c}_j$.
 - 10: **end if**
 - 11: **end for**
 - 12: **return** $\mathbf{Y}$.
-

Algorithm 5 Decode($\mathbf{W}$)

Require: Noisy message matrix $\mathbf{W} \in \mathbb{Z}_q^{\ell_r \times \ell_s}$.

- 1: Initialize an empty bit string M' .
 - 2: Set $N_{\text{blk}} \leftarrow \ell_r \ell_s / 8$.
 - 3: **for** $j = 0, \dots, N_{\text{blk}} - 1$ **do**
 - 4: **if** $\ell_r = 8$ **then**
 - 5: $\mathbf{w}_j \leftarrow \mathbf{W}_{*,j}$.
 - 6: **else**
 - 7: $\mathbf{w}_j \leftarrow \mathbf{W}_{j,*}$.
 - 8: **end if**
 - 9: $\hat{\mathbf{c}}_j \leftarrow \mathbf{Q}_{\alpha_{E_8} E_8}(\mathbf{w}_j) \pmod{q}$.
 - 10: $(a_{j,0}, \dots, a_{j,7}) \leftarrow \text{Delabel}_{E_8, \mathbf{b}}(\hat{\mathbf{c}}_j)$.
 - 11: Append the b_h -bit representation of each $a_{j,h}$, for $h = 0, \dots, 7$, to M' .
 - 12: **end for**
 - 13: **return** M' .
-

The internal maps $\text{Label}_{E_8, \mathbf{b}}$ and $\text{Delabel}_{E_8, \mathbf{b}}$ used by `Encode` and `Decode` are specified in Section 9.8.

2.3 Frost KEM

`Frost.KEM` is obtained from `Frost.PKE` by a Fujisaki–Okamoto-style transform with implicit rejection. The KEM secret key stores the PKE secret key, the public key, the public-key hash, and a profile-dependent fallback string. Encapsulation samples the FO plaintext, derives the PKE encryption coins and intermediate key material from this plaintext and the public-key hash, and then calls the deterministic PKE encryption algorithm. Decapsulation decrypts the ciphertext, re-derives the encryption coins from the recovered message, re-encrypts, and derives the fallback key if ciphertext validation fails.

The value r_{Enc} output by $\mathcal{G}_{\text{FO}}$ is parsed as (σ, μ) . Here σ determines the ephemeral secret matrix $\mathbf{R}$, while μ determines the ciphertext-side dithers $(\mathbf{D}_u, \mathbf{D}_v)$. The seed μ is included in the ciphertext so that decryption and re-encryption use the same public dithers.

Algorithm 6 `Frost.KEM.KeyGen()`

```

1:  $z \leftarrow \mathcal{U}(\{0, 1\}^{\kappa_{\text{ss}}})$ .
2:  $(\text{pk}, \text{sk}_{\text{PKE}}) \leftarrow \text{Frost.PKE.KeyGen}()$ .
3:  $h_{\text{pk}} \leftarrow \mathcal{H}_{\text{pk}}(\text{pk})$ .
4:  $\text{sk}_{\text{KEM}} \leftarrow (\text{sk}_{\text{PKE}}, \text{pk}, h_{\text{pk}}, z)$ .
5: return  $(\text{pk}, \text{sk}_{\text{KEM}})$ .

```

Algorithm 7 `Frost.KEM.Encaps(pk)`

Require: Public key pk .

```

1:  $h_{\text{pk}} \leftarrow \mathcal{H}_{\text{pk}}(\text{pk})$ .
2:  $M \leftarrow \mathcal{U}(\mathcal{M}_{\text{FO}})$ .
3:  $(r_{\text{Enc}}, K') \leftarrow \mathcal{G}_{\text{FO}}(h_{\text{pk}} \parallel M)$ .
4:  $\text{ct} \leftarrow \text{Frost.PKE.Enc}(\text{pk}, M, r_{\text{Enc}})$ .
5:  $K \leftarrow \mathcal{H}_K(\mathcal{H}_{\text{ct}}(\text{ct}) \parallel K')$ .
6: return  $(\text{ct}, K)$ .

```

Algorithm 8 `Frost.KEM.Decaps(sk_KEM, ct)`

Require: Secret key $\text{sk}_{\text{KEM}} = (\text{sk}_{\text{PKE}}, \text{pk}, h_{\text{pk}}, z)$ and ciphertext ct .

```

1:  $M' \leftarrow \text{Frost.PKE.Dec}(\text{sk}_{\text{PKE}}, \text{ct})$ .
2:  $(r'_{\text{Enc}}, K') \leftarrow \mathcal{G}_{\text{FO}}(h_{\text{pk}} \parallel M')$ .
3:  $\text{ct}' \leftarrow \text{Frost.PKE.Enc}(\text{pk}, M', r'_{\text{Enc}})$ .
4: if  $\text{ct}' = \text{ct}$  then
5:    $K \leftarrow \mathcal{H}_K(\mathcal{H}_{\text{ct}}(\text{ct}) \parallel K')$ .
6: else
7:    $K \leftarrow \mathcal{H}_K(\mathcal{H}_{\text{ct}}(\text{ct}) \parallel z)$ .
8: end if
9: return  $K$ .

```

The branch in `Frost.KEM.Decaps` specifies the implicit-rejection functionality. A conforming implementation computes both candidate keys and performs the final selection in constant time. The equality test compares the full ciphertext, including the split ciphertext components and the public seed μ . Once (pk, M) is fixed, the coin string r_{Enc} is fixed by $\mathcal{G}_{\text{FO}}(h_{\text{pk}} \parallel M)$, and hence $\text{Frost.PKE.Enc}(\text{pk}, M, r_{\text{Enc}})$ is deterministic. For all profiles and variants considered in this document, $\kappa_{\text{m}} = \kappa_{\text{ss}} = \lambda$. Consequently, both the final shared secret and the implicit-rejection fallback string have λ bits for the selected profile.

3 Parameter Settings

This section summarizes the submitted **Frost** parameter sets and the compact-ciphertext variants used for the high-security profiles. The five default profiles are **Frost-128**, **Frost-192**, **Frost-256**, **Frost-384**, and **Frost-512**. Each profile fixes the matrix dimensions $n = m$, the message dimensions (ℓ_r, ℓ_s) , the modulus q , the centered-binomial parameters (η_s, η_r) , the message bit-width b_{msg} , and the compression widths $(t_{\text{pk}}, t_u, t_v)$.

The long-term secret matrix has dimension $n \times \ell_r$, and the compact public-key matrix $\mathbf{B}$ has dimension $m \times \ell_r$. The ephemeral secret matrix has dimension $m \times \ell_s$, while the compact ciphertext component $\mathbf{U}$ has dimension $n \times \ell_s$. The message matrix and the compact ciphertext component $\mathbf{V}$ both have dimension $\ell_r \times \ell_s$. For all submitted profiles, the number of FO plaintext bits is $b_{\text{msg}}\ell_r\ell_s = \lambda$.

The default profiles use $\ell_r = 8$, so that the message matrix is decoded column-wise. For **Frost-384** and **Frost-512**, we also provide compact-ciphertext variants that exchange ℓ_r and ℓ_s . This keeps $b_{\text{msg}}\ell_r\ell_s$, $n = m$, q , (η_s, η_r) , and $(t_{\text{pk}}, t_u, t_v)$ unchanged, while moving part of the transmission cost from the ciphertext to the public key. The variants are useful when a public key is cached or certified once and then used for many encapsulations.

3.1 Default Parameter Sets

Table 3 gives the selected parameters, external byte sizes, decryption-failure estimates, and concrete security estimates for the five default **Frost** profiles. The dimension $n = m$ grows from 512 to 2600 to scale security. The arithmetic modulus is $q = 2^{15}$ for **Frost-128** and $q = 2^{16}$ for all higher profiles. The row dimension $\ell_r = 8$ fixes the 8-dimensional message layer, while ℓ_s increases for the higher-security profiles to keep $b_{\text{msg}}\ell_r\ell_s = \lambda$. The shared-secret length $|\text{ss}|$ is exactly $\lambda/8$ bytes.

Table 3: Summary of selected parameters, byte sizes, correctness, and concrete security estimates for the default **Frost** profiles. Sizes are in bytes and security estimates are in bits.

Metric	Frost-128	Frost-192	Frost-256	Frost-384	Frost-512
Core Parameters					
λ	128	192	256	384	512
$n = m$	512	880	1288	1928	2600
(ℓ_r, ℓ_s)	(8, 8)	(8, 8)	(8, 8)	(8, 12)	(8, 16)
q	2^{15}	2^{16}	2^{16}	2^{16}	2^{16}
(η_s, η_r)	(2, 2)	(1, 1)	(1, 1)	(1, 1)	(1, 1)
b_{msg}	2	3	4	4	4
$(t_{\text{pk}}, t_u, t_v)$	(10, 10, 5)	(11, 11, 6)	(13, 12, 8)	(13, 13, 9)	(14, 14, 7)
Byte Sizes					
$ \text{pk} $	5152	9712	16776	25096	36432
$ \text{ct} $	5192	9760	15552	37736	72944
$ \text{sk}_{\text{KEM}} $	6736	11528	19416	29032	41728
$ \text{ss} $	16	24	32	48	64
Correctness and Concrete Security					
$\log_2 \delta_{\text{dec}}$	-169.44	-199.84	-266.61	-440.52	-581.12
MATZOV	131.85	197.65	257.08	393.79	512.97
CORESVP-C	107.75	175.27	236.36	376.83	498.68
CORESVP-Q	98.58	161.13	216.55	345.16	456.47

The concrete byte sizes are derived from the packed public-key, ciphertext, and secret-key layouts. The public key is $\text{pk} = (\rho, \mathbf{B})$, where ρ occupies 32 bytes and $\mathbf{B} \in \mathbb{Z}_{p_{\text{pk}}}^{m \times \ell_r}$ is packed with t_{pk} bits per entry. The ciphertext is $\text{ct} = (\mu, \mathbf{U}, \mathbf{V})$, where μ occupies 32 bytes, $\mathbf{U} \in \mathbb{Z}_{p_u}^{n \times \ell_s}$ is packed with t_u bits per entry, and $\mathbf{V} \in \mathbb{Z}_{p_v}^{\ell_r \times \ell_s}$ is packed with t_v bits per entry. Thus

$$|\text{pk}| = 32 + \left\lceil \frac{m\ell_r t_{\text{pk}}}{8} \right\rceil, \quad |\text{ct}| = 32 + \left\lceil \frac{n\ell_s t_u + \ell_r \ell_s t_v}{8} \right\rceil.$$

The KEM secret key stores the PKE secret key, the public key, the public-key hash h_{pk} , and the implicit-rejection fallback string z . In the external byte-size accounting used in Table 3, the PKE secret key is the packed long-term secret matrix $\mathbf{S}$. Since $\mathbf{S} \leftarrow \mathcal{B}_{\eta_s}^{n \times \ell_r}$, each entry of $\mathbf{S}$ is represented using

$$t_s = \lceil \log_2(2\eta_s + 1) \rceil$$

bits. The public-key hash h_{pk} is a 32-byte string, while the fallback string z has length $\kappa_{\text{ss}}/8$ bytes. Consequently,

$$|\text{sk}_{\text{KEM}}| = |\text{pk}| + \left\lceil \frac{n\ell_r t_s}{8} \right\rceil + 32 + \frac{\kappa_{\text{ss}}}{8}.$$

For Frost-128, $\eta_s = 2$, hence $t_s = 3$. For the higher profiles, $\eta_s = 1$, hence $t_s = 2$. This gives the secret-key sizes reported in Table 3.

The decryption-failure entries in Table 3 are whole-ciphertext estimates for the message layer. The concrete security entries are computed from the LWE normal forms induced by the dithered-quantization transcript. For each profile, the reported security value is the minimum over the relevant public-key and ciphertext-side LWE anchors and the attack families considered by the Lattice Estimator. The MATZOV estimate is the main concrete estimator output, while the CoreSVP columns give classical and quantum reduction-cost baselines.

3.2 Compact-Ciphertext Variants

For the two highest security levels, Frost also provides compact-ciphertext variants. These variants keep the same $n = m$, q , (η_s, η_r) , b_{msg} , and compression widths as the corresponding default profiles. They only exchange the two message dimensions ℓ_r and ℓ_s . Since $b_{\text{msg}}\ell_r\ell_s = \lambda$, the FO plaintext length is unchanged.

The transformation changes the relative sizes of the public key and ciphertext. The public key scales with $m\ell_r t_{\text{pk}}$, while the dominant ciphertext component scales with $n\ell_s t_u$. For Frost-384 and Frost-512, we have $n = m$ and $t_{\text{pk}} = t_u$. Exchanging ℓ_r and ℓ_s therefore preserves the total $|\text{pk}| + |\text{ct}|$, but shifts bytes from the ciphertext to the public key.

Table 4: Compact-ciphertext variants for the high-security Frost profiles. Sizes are in bytes.

Profile	(ℓ_r, ℓ_s)	$ \text{pk} $	$ \text{ct} $	$ \text{pk} + \text{ct} $	Ciphertext reduction
Frost-384	(8, 12)	25096	37736	62832	–
Frost-CC-384	(12, 8)	37628	25204	62832	12532 B (33.2%)
Frost-512	(8, 16)	36432	72944	109376	–
Frost-CC-512	(16, 8)	72832	36544	109376	36400 B (49.9%)

The compact-ciphertext variants do not introduce a new hardness assumption or a new KEM transform. They use the same dithered-quantization rule, the same FO flow, and the same LWE normal-form security argument as the corresponding default profiles. The decryption-error matrix remains

$$\mathbf{E}_{\text{dec}} = \mathbf{E}_{\text{pk}}^T \mathbf{R} - \mathbf{S}^T \mathbf{E}_u + \mathbf{E}_v.$$

In the default high-security profiles, decoding is applied to the columns because $\ell_r = 8$. In the compact-ciphertext variants, decoding is applied to the rows because $\ell_s = 8$. Since $n = m$, $\eta_s = \eta_r$, and $t_{\text{pk}} = t_u$ for these profiles, the correctness and concrete-security analyses are the same up to this transposition.

For levels 128, 192, and 256, the default profiles already have $(\ell_r, \ell_s) = (8, 8)$. The dimension-exchange transformation is therefore the identity at these levels and does not produce a distinct compact-ciphertext layout.

4 Known Answer Tests

The official NGCC known-answer tests for Frost are included under `API_PKC/Test_Vectors/` in the submission package. The submitted `API_PKC` package contains the Frost default instances and the compact-ciphertext variant instances. Each instance is provided under both the Reference and Optimized implementation trees.

The relevant package layout is as follows:

```
API_PKC/
  Implementations/
    Reference_Implementation/
      MAMBA-Frost-128/
      MAMBA-Frost-192/
      MAMBA-Frost-256/
      MAMBA-Frost-384/
      MAMBA-Frost-512/
      MAMBA-Frost-CC-128/
      MAMBA-Frost-CC-192/
      MAMBA-Frost-CC-256/
      MAMBA-Frost-CC-384/
      MAMBA-Frost-CC-512/
    Optimized_Implementation/
      MAMBA-Frost-128/
      MAMBA-Frost-192/
      MAMBA-Frost-256/
      MAMBA-Frost-384/
      MAMBA-Frost-512/
      MAMBA-Frost-CC-128/
      MAMBA-Frost-CC-192/
      MAMBA-Frost-CC-256/
      MAMBA-Frost-CC-384/
      MAMBA-Frost-CC-512/
  Test_Vectors/
```

Each instance directory contains the NGCC `API_PKC` KEM interface files, the official KAT generator, and the auxiliary support files required by the interface. In particular, the files `KEM_AlgorithmInstance.h`, `KEM_AlgorithmInstance.c`, `KAT_KEM.c`, `drng.c`, `drng.h`, `auxfunc.c`, and `auxfunc.h` are provided for each submitted instance. The files `KAT_KEM.c`, `drng.c`, `drng.h`, `auxfunc.c`, and `auxfunc.h` follow the official `API_PKC` testing interface.

For each parameter set, running `make kat-generate` in a Reference or Optimized instance directory builds the API instance, executes the official KAT generator, and writes an instance-local output file. The target `make kat-install` copies the generated KAT file to `API_PKC/Test_Vectors/`. The target `make kat-repro` checks the instance-local output against the corresponding formal KAT file. The Reference output is used as the formal KAT value, and the Optimized output is required to be byte-identical to the Reference output.

Table 5: Official NGCC `API_PKC` KAT files for the submitted Frost instances.

Instance	pk	sk	ct	ss	KAT file
MAMBA-Frost-128	5152	6736	5192	16	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-128.txt
MAMBA-Frost-192	9712	11528	9760	24	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-192.txt
MAMBA-Frost-256	16776	19416	15552	32	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-256.txt
MAMBA-Frost-384	25096	29032	37736	48	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-384.txt
MAMBA-Frost-512	36432	41728	72944	64	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-512.txt
MAMBA-Frost-CC-128	5152	6736	5192	16	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-CC-128.txt
MAMBA-Frost-CC-192	9712	11528	9760	24	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-CC-192.txt
MAMBA-Frost-CC-256	16776	19416	15552	32	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-CC-256.txt
MAMBA-Frost-CC-384	37628	43492	25204	48	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-CC-384.txt
MAMBA-Frost-CC-512	72832	83328	36544	64	API_PKC/Test_Vectors/KAT_KEM_MAMBA-Frost-CC-512.txt

Each KAT file consists of a sequence of deterministic test records. A record contains the deterministic test seed, the generated public key, secret key, ciphertext, and shared secret, together with their byte lengths. The fields are written as `Count`, `Seed_Len`, `Seed`, `PK_Len`, `PK`, `SK_Len`, `SK`, `CT_Len`, `CT`, `SS_Len`, and `SS`. The byte lengths in the KAT records match the external API lengths reported in Table 5.

The compact-ciphertext instances `MAMBA-Frost-CC-384` and `MAMBA-Frost-CC-512` use the transposed message layout described in Section 3.2. The `MAMBA-Frost-CC-128`, `MAMBA-Frost-CC-192`, and `MAMBA-Frost-CC-256` instance directories are included to keep a uniform API and KAT structure across security levels; their external sizes match the corresponding default instances.

Consequently, the submitted KAT files are produced through the same `NGCC_API_PKC` interface used for conformance testing. The Reference and Optimized implementations expose the same public API, produce byte-identical KAT outputs for the same deterministic test seeds, and use the same external key, ciphertext, and shared-secret lengths.

5 Efficiency Evaluation

The submission package contains a portable reference implementation and an AVX2 implementation. The reference implementation is written in ISO C99 and serves as the baseline for specification matching, Known Answer Test generation, API validation, and functional testing. The AVX2 implementation preserves the same algorithmic structure and external byte layout, while accelerating matrix arithmetic, packing, quantization, and related inner loops with vector instructions. Both implementations follow the same submitted algorithms, byte encodings, message embedding, and Fujisaki–Okamoto flow.

This section reports the performance of the submitted `API_PKC` package through the official KEM interface and the official auxiliary-function path. The benchmarked functions are the API-level routines `KeyGen`, `Encaps`, and `Decaps`, as exposed by the submitted implementation interface. The native implementation benchmark, which uses the implementation’s own low-level expansion backends and is intended for engineering comparison with optimized KEM implementations, is reported separately in Appendix C.

5.1 Benchmarking Platforms

Table 6 summarizes the benchmarking environments used for the x86 measurements and the Cortex-M4 measurements. The x86-64 platform is used for both the `API_PKC` self-evaluation and the native x86 benchmark in Appendix C. The Cortex-M4 measurements are reported as an additional embedded-platform benchmark and are not part of the x86 `API_PKC` self-evaluation path.

5.2 x86-64 `API_PKC` Self-Evaluation Results

Table 7 reports the x86-64 self-evaluation results for the submitted `API_PKC` package. The measurements include the submitted KEM wrapper layer, the `API_PKC` DRNG interface used by the KAT and test harnesses, and the official auxiliary-function path used for XOF calls. They therefore represent the conformance-oriented `API_PKC` performance profile, rather than the native low-level implementation profile reported in Appendix C.

The compact-ciphertext entries are `Frost` instances using the `MAMBA-Frost-CC-*` API instance names. They are reported as implementation variants of the same algorithm, not as a separate algorithm. For security levels 128, 192, and 256, the compact-ciphertext instance names use the same external layout as the corresponding default instances and are omitted from the table. Only the two high-security compact-ciphertext instances are listed separately.

The optimized implementation substantially reduces the API-level cycle counts across all profiles. The largest speedups occur in matrix-dominated operations. The Reference results are comparatively high because the `API_PKC` self-evaluation path includes the submitted wrapper layer and the official auxiliary-function path, rather than the native low-level matrix-expansion backend.

Table 6: Benchmarking environments for the **Frost** efficiency evaluation.

Item	x86-64 platform	Cortex-M4 platform
Target	AMD Ryzen 7 9700X 8-Core Processor, x86-64, with AVX2 and AES-NI enabled.	STM32 Nucleo-L4R5ZI-P board with STM32L4R5ZIT6P MCU, Arm Cortex-M4F, 2 MB Flash, and 640 KB SRAM; pqm4 platform nucleo-l4r5zi.
Operating environment	Ubuntu 22.04.5 LTS, Linux 6.8.0-124-generic.	WSL1 Ubuntu 22.04.5 LTS host with STM32CubeProgrammer CLI 2.22.0.
Compiler and tools	gcc 11.4.0; GNU Make 4.3; CMake 3.22.1.	arm-none-eabi-gcc 10.3.1; GNU size 2.38; Python 3.10.12; GNU Make 4.3.
Build options	Reference: -O2 -std=c99 -Wpedantic -Wall -Wextra. Optimized: -O3 -march=x86-64 -mavx2 -maes -mtune=native -flto -fomit-frame-pointer -std=c99.	pqm4 -opt speed; -O3 -g3; Cortex-M4F flags -mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16; LTO disabled by default.
Timing	Serialized x86 cycle counter and CLOCK_MONOTONIC_RAW; single-threaded execution; 100 warm-up runs and 100 measured runs.	20 MHz for pqm4 CLOCK_BENCHMARK; 120 MHz for CLOCK_FAST; cycle counts measured by the on-device SysTick timer.
Validation and metrics	API_PKC build, smoke tests, correctness checks, KAT generation, KAT reproducibility, size checks, dependency checks, and artifact-cleanliness checks all passed. Cycle counts are reported in kCycles.	Cycle counts are averages over 10 executions and reported in kCycles; peak stack usage is measured by the pqm4 stack benchmark; binary size is measured by arm-none-eabi-size.

Table 7: x86-64 API_PKC self-evaluation results for **Frost** in kCycles.

Instance	Reference			Optimized		
	KeyGen	Encaps	Decaps	KeyGen	Encaps	Decaps
Frost-128	37,270	40,204	39,021	966	1,560	2,104
Frost-192	106,905	124,595	110,709	2,262	2,910	3,874
Frost-256	229,887	234,669	235,626	4,046	5,613	7,390
Frost-384	515,015	560,493	562,001	7,887	13,555	16,465
Frost-CC-384	528,168	529,000	528,756	10,905	11,366	14,634
Frost-512	932,777	1,111,746	1,112,164	13,740	27,944	34,348
Frost-CC-512	985,684	976,594	989,677	23,841	21,447	28,721

5.3 Cortex-M4 Efficiency Results

Tables 8 and 9 report the Cortex-M4 results for **Frost**. These measurements are reported as an additional embedded-platform benchmark. Both AES128 and SHAKE128 matrix-expansion backends are included. The AES128 backend is consistently faster, while the SHAKE128 backend provides an XOF-only alternative.

On Cortex-M4, expansion and hashing dominate the lower-security profiles. For **Frost-384** and **Frost-512**, the hashing share decreases during encapsulation and decapsulation, indicating that dense arithmetic and memory movement account for a larger fraction of the total cycle count. The peak stack usage remains within the SRAM budget of the target board for all measured profiles.

6 Correctness and Security Analysis

This section states the correctness and security claims for **Frost**. The detailed probability computation and proof details are given in Appendix B. The main text first explains the lattice-code layer and the resulting decryption-failure bound, and then states the theoretical and concrete security claims for **Frost.PKE** and **Frost.KEM**.

Table 8: Cortex-M4 cycle counts for **Frost**. Values are averages over 10 executions and are reported in kCycles.

Scheme	Backend	KeyGen	Encaps	Decaps
Frost-128	AES128	32,966	32,284	33,399
	SHAKE128	48,206	48,885	50,000
Frost-192	AES128	94,968	92,835	94,903
	SHAKE128	132,015	132,361	134,429
Frost-256	AES128	202,600	194,903	197,704
	SHAKE128	276,878	277,820	280,621
Frost-384	AES128	476,770	807,286	812,997
	SHAKE128	615,644	994,489	1,000,200
Frost-CC-384	AES128	524,210	704,829	709,581
	SHAKE128	666,792	843,730	848,482
Frost-512	AES128	869,619	1,715,704	1,729,487
	SHAKE128	1,121,003	1,960,325	1,974,108
Frost-CC-512	AES128	1,034,489	1,229,311	1,238,255
	SHAKE128	1,285,873	1,473,922	1,482,866

6.1 Security Definitions and Claims

Public Key Encryption. A public key encryption scheme **PKE** consists of a tuple of polynomial-time algorithms (**KeyGen**, **Enc**, **Dec**). The key-generation algorithm **KeyGen**(1^λ) outputs a public key **pk** and a secret key **sk**. The encryption algorithm **Enc**(**pk**, m) takes $m \in \mathcal{M}$ and outputs a ciphertext **ct**. The decryption algorithm **Dec**(**sk**, **ct**) outputs a message m' or a failure symbol $\perp$.

δ -Correctness. A **PKE** scheme is δ -correct if for all $(\mathbf{pk}, \mathbf{sk}) \leftarrow \mathbf{KeyGen}(1^\lambda)$ and all $m \in \mathcal{M}$,

$$\Pr[\mathbf{Dec}(\mathbf{sk}, \mathbf{Enc}(\mathbf{pk}, m)) \neq m] \leq \delta.$$

The probability is taken over the randomness of **Enc**. For **Frost.PKE**, the target regime is a cryptographically negligible decryption-failure probability.

IND-CPA Security. For a **PKE** adversary $\mathcal{A}$, define

$$\mathbf{Adv}_{\mathbf{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr \left[\mathcal{A}^{\mathbf{Enc}(\mathbf{pk}, \cdot)}(\mathbf{pk}, \mathbf{ct}^*) = b \right] - \frac{1}{2} \right|,$$

where $(\mathbf{pk}, \mathbf{sk}) \leftarrow \mathbf{KeyGen}$, the adversary chooses m_0, m_1 , the challenger samples $b \leftarrow \mathcal{U}(\{0, 1\})$, and returns $\mathbf{ct}^* \leftarrow \mathbf{Enc}(\mathbf{pk}, m_b)$.

Key Encapsulation Mechanism. A **KEM** scheme **KEM** consists of a tuple of algorithms (**KeyGen**, **Encaps**, **Decaps**). The key-generation algorithm **KeyGen**(1^λ) outputs a public key **pk** and a secret key **sk**. The encapsulation algorithm **Encaps**(**pk**) outputs a ciphertext **ct** and a shared key $K \in \mathcal{K}$. The decapsulation algorithm **Decaps**(**sk**, **ct**) outputs a shared key $K' \in \mathcal{K}$. In implicit-rejection **KEMs** such as **Frost.KEM**, malformed ciphertexts are mapped to a pseudorandom fallback key rather than to an externally visible failure symbol.

IND-CCA Security. For a **KEM** adversary $\mathcal{A}$ with access to a decapsulation oracle $\mathcal{O}_{\text{dec}}$, define

$$\mathbf{Adv}_{\mathbf{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr \left[\begin{array}{l} (\mathbf{pk}, \mathbf{sk}) \leftarrow \mathbf{KeyGen}(1^\lambda), \quad b \leftarrow \mathcal{U}(\{0, 1\}), \\ (\mathbf{ct}^*, K_0) \leftarrow \mathbf{Encaps}(\mathbf{pk}), \quad K_1 \leftarrow \mathcal{U}(\mathcal{K}), \quad : b' = b \\ b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{dec}}}(\mathbf{pk}, \mathbf{ct}^*, K_b) \end{array} \right] - \frac{1}{2} \right|.$$

The adversary is not allowed to query $\mathcal{O}_{\text{dec}}$ on $\mathbf{ct}^*$.

Claim 6.1 (IND-CCA Security in ROM). *The **Frost.KEM** construction is IND-CCA secure in the classical random-oracle model under the hardness of the LWQ problem.*

Table 9: Cortex-M4 resource profile for Frost. Memory and binary size are reported in bytes. Hashing share is reported in percent. Binary size denotes the total linked size. For these entries, the corresponding .data and .bss sections are zero.

Scheme	Backend	Peak stack			Hashing share			Binary size
		KeyGen	Encaps	Decaps	KeyGen	Encaps	Decaps	
Frost-128	AES128	25,884	64,276	72,660	74.5%	78.8%	78.4%	11,236
	SHAKE128	21,348	55,644	64,028	83.7%	84.3%	83.9%	11,360
Frost-192	AES128	43,588	110,124	124,404	75.1%	78.9%	78.7%	11,756
	SHAKE128	36,108	95,604	109,884	82.2%	83.4%	83.2%	11,848
Frost-256	AES128	63,764	161,676	182,508	74.5%	79.0%	78.8%	13,444
	SHAKE128	53,028	140,628	161,460	82.6%	83.4%	83.3%	13,540
Frost-384	AES128	70,876	133,276	179,964	70.2%	42.5%	42.7%	6,296
	SHAKE128	66,580	128,972	175,660	79.7%	50.1%	50.2%	6,340
Frost-CC-384	AES128	101,724	117,852	195,380	64.5%	48.5%	48.6%	6,216
	SHAKE128	97,420	113,548	191,076	74.1%	58.9%	59.0%	6,264
Frost-512	AES128	95,164	179,228	179,492	69.7%	36.2%	36.4%	6,484
	SHAKE128	89,516	173,572	173,836	79.2%	46.1%	46.1%	6,528
Frost-CC-512	AES128	178,364	96,020	179,500	59.5%	50.3%	50.3%	6,840
	SHAKE128	172,708	90,364	173,844	69.8%	61.1%	61.0%	6,900

Claim 6.2 (IND-CCA Security in QROM). *The Frost.KEM construction is IND-CCA secure in the quantum random-oracle model under the hardness of the LWQ problem.*

Claim 6.3 (Concrete security estimates). *The five submitted Frost profiles target nominal security levels 128, 192, 256, 384, and 512. Table 3 reports the corresponding concrete lattice estimates under three models: the MATZOV classical model, the Core-SVP classical baseline, and the Core-SVP quantum sieve model. The MATZOV classical estimates exceed the nominal targets for all five profiles; the Core-SVP estimates are reported separately as conservative baseline and quantum comparison values.*

6.2 Correctness of the PKE Scheme

Frost embeds the FO plaintext through an 8-dimensional lattice-code layer [Zam14, MSC10, LLL⁺24b]. Let $\alpha_{E_8} = q/2^{b_{\text{msg}}}$. Each block of $8b_{\text{msg}}$ message bits is mapped by a fixed public labeling map to a point in the finite codebook $\mathcal{C}_{E_8} = \alpha_{E_8} E_8 \cap [0, q)^8$. Decoding applies nearest-neighbor decoding in the scaled lattice $\alpha_{E_8} E_8$, followed by the inverse labeling map. This message layer is used only for encoding and correctness; it does not change the LWQ transcript or the LWE normal forms used in the security proof.

For the default profiles, $\ell_r = 8$, and the message matrix is processed column-wise. Each column of the decoded matrix is one E_8 -coded block, and the number of decoding blocks is ℓ_s . For the compact-ciphertext variants, $\ell_s = 8$, and the same E_8 code is applied row-wise. Each row of the decoded matrix is then one E_8 -coded block, and the number of decoding blocks is ℓ_r . The compact-ciphertext variants at the 384- and 512-bit security levels are obtained by transposing the two message dimensions. Since $n = m$, $\eta_s = \eta_r$, and $t_{\text{pk}} = t_u$, equivalently $p_{\text{pk}} = p_u$, the column-wise error distribution of the default profile and the row-wise error distribution of the compact-ciphertext variant are identical after transposition. Consequently, the E_8 decoding-block distribution and the whole-ciphertext DFR bound are preserved.

For $x \in \{\text{pk}, u, v\}$, write $\chi_x = \chi_{q, p_x}$ for the scalar error distribution induced by dithered quantization with modulus pair (q, p_x) , and let $\Delta_x = q/p_x$. For the power-of-two parameter sets used by Frost, $e_Q \leftarrow \chi_x$ is uniform over $\{-\Delta_x/2+1, \dots, \Delta_x/2\}$. Thus $\mathbb{E}[e_Q] = 1/2$, $\text{Var}(e_Q) = (\Delta_x^2 - 1)/12$, and $\mathbb{E}[e_Q^2] = (\Delta_x^2 + 2)/12$. The quantization error is independent of the corresponding quantizer input [Zam14, LLL24a].

After public dither compensation, the reconstructed public-key and ciphertext components have the normal forms $\hat{\mathbf{B}} = \mathbf{A}\mathbf{S} + \mathbf{E}_{\text{pk}}$, $\hat{\mathbf{U}} = \mathbf{A}^T \mathbf{R} + \mathbf{E}_u$, $\hat{\mathbf{V}} = \hat{\mathbf{B}}^T \mathbf{R} + \text{Encode}(M) + \mathbf{E}_v$. Here the entries

of $\mathbf{E}_{\text{pk}}$, $\mathbf{E}_{\text{u}}$, and $\mathbf{E}_{\text{v}}$ follow χ_{pk} , χ_{u} , and χ_{v} , respectively. Decryption computes $\mathbf{W} = \hat{\mathbf{V}} - \mathbf{S}^\top \hat{\mathbf{U}}$, and hence

$$\mathbf{W} = \text{Encode}(M) + \mathbf{E}_{\text{dec}}, \quad \mathbf{E}_{\text{dec}} = \mathbf{E}_{\text{pk}}^\top \mathbf{R} - \mathbf{S}^\top \mathbf{E}_{\text{u}} + \mathbf{E}_{\text{v}}.$$

All terms in $\mathbf{E}_{\text{dec}}$ lie in $\mathbb{Z}_q^{\ell_r \times \ell_s}$. Correct decoding holds whenever every 8-dimensional error block remains inside the Voronoi region of $\alpha_{E_8} E_8$, with the fixed tie convention used by the decoder [MSC10, LLL⁺24b].

A scalar approximation gives a useful closed-form view of the correctness margin. Using $\text{Var}(\mathcal{B}_\eta) = \eta/2$ and the second moments of the quantization errors, one scalar entry of $\mathbf{E}_{\text{dec}}$ has centered variance

$$\sigma_{\text{tot}}^2 = m \cdot \frac{\eta_r}{2} \cdot \frac{\Delta_{\text{pk}}^2 + 2}{12} + n \cdot \frac{\eta_s}{2} \cdot \frac{\Delta_{\text{u}}^2 + 2}{12} + \frac{\Delta_{\text{v}}^2 - 1}{12}.$$

This is the analogue of the total-noise variance used in lattice-code DFR estimates for LWE-based schemes [LLL⁺24b, WZZ⁺24], with the explicit sampled-error terms replaced by the quantization-error terms arising from the LWQ transcript. Let $\mathbf{g} \leftarrow \mathcal{D}_{\mathbb{R}^8, \sigma_{\text{tot}}}$ denote an 8-dimensional continuous Gaussian approximation to one centered decoding block. The corresponding block-failure approximation is

$$\delta_{\text{dec}}^{\text{G}} \approx 1 - (1 - p_{E_8}^{\text{G}})^{N_{\text{blk}}},$$

where $p_{E_8}^{\text{G}} = \Pr[\mathbf{g} \notin \alpha_{E_8} \text{Vor}(E_8)]$, $N_{\text{blk}} = \ell_s$ for the default profiles and $N_{\text{blk}} = \ell_r$ for the compact-ciphertext variants. Equivalently, for the very small failure probabilities targeted here, one may use the union approximation $\delta_{\text{dec}}^{\text{G}} \approx N_{\text{blk}} p_{E_8}^{\text{G}}$.

While the Gaussian heuristic yields a transparent dependence of the DFR on system parameters, it is less accurate in the distribution tails. Therefore, all decryption-failure values reported in the tables are computed using the exact finite-support root-projection method described in Appendix B.1. This method evaluates the E_8 Voronoi-facet tail probabilities directly from the PMFs of $\mathcal{B} * \eta_s$, $\mathcal{B} * \eta_r$, χ_{pk} , χ_{u} , and χ_{v} , combined with a full-ciphertext union bound over all E_8 decoding blocks. Throughout this work, δ_{dec} denotes this rigorous upper bound, which dominates the true decryption-failure probability and is used as the correctness-loss term in the FO security analysis.

6.3 Theoretical Security

The security argument uses the LWE normal forms induced by public dithered quantization [Reg05, BLP⁺13, LLL24a]. The split representation used by Frost is a public encoding of the same normal-form sample, so the split and normal views preserve distinguishing advantage.

Theorem 1. *In the divisible power-of-two setting used by Frost, the LWQ normal-form distribution with secret distribution χ_s is identical to the LWE distribution with secret distribution χ_s and error distribution $\chi_{q,p}$. Moreover, the split representation and the normal representation are related by a public coordinate-wise bijection, and therefore their decision experiments have the same distinguishing advantage.*

For matrix-width LWE instances, write $\text{Adv}_{q,n,m,w,\chi_s,\chi_e}^{\text{LWE}}(\cdot)$ for decisional LWE over $\mathbb{Z}_q^{m \times n}$ with width w , secret distribution χ_s , and error distribution χ_e [Reg05]. The notation also permits row-wise mixed error distributions.

Theorem 2 (IND-CPA security of Frost.PKE). *For every PPT IND-CPA adversary $\mathcal{A}$ against the seeded implementation of Frost.PKE, there exist PPT adversaries $\mathcal{C}_{\text{Exp}}$, $\mathcal{B}_1$, and $\mathcal{B}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{Frost.PKE}}^{\text{IND-CPA}}(\mathcal{A}) &\leq \text{Adv}_{\text{Exp}}(\mathcal{C}_{\text{Exp}}) + \text{Adv}_{q,n,m,\ell_r,\mathcal{B}_{\eta_s},\chi_{q,p_{\text{pk}}}}^{\text{LWE}}(\mathcal{B}_1) \\ &\quad + \text{Adv}_{q,m,n+\ell_r,\ell_s,\mathcal{B}_{\eta_r},(\chi_{q,p_u}^n, \chi_{q,p_v}^{\ell_r})}^{\text{LWE}}(\mathcal{B}_2). \end{aligned}$$

The first LWE term is the public-key anchor. The second LWE term is the joint ciphertext anchor, where $\mathbf{U}$ and $\mathbf{V}$ are replaced together because both are generated from the same ephemeral secret $\mathbf{R}$. The proof is given in Appendix B.3.

Theorem 3 (Classical ROM). *Assume that $\mathcal{G}$ and $\mathcal{H}$ are random oracles. For every classical adversary $\mathcal{A}$ against the IND-CCA security of Frost.KEM making at most $q_{\mathcal{G}}$ queries to $\mathcal{G}_{\text{FO}}$, at most $q_{\mathcal{H}}$ total queries to the domain-separated instances of $\mathcal{H}$, and at most q_{dec} decapsulation queries, there exists an IND-CPA adversary $\mathcal{B}$ against Frost.PKE such that*

$$\text{Adv}_{\text{Frost.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 3\text{Adv}_{\text{Frost.PKE}}^{\text{IND-CPA}}(\mathcal{B}) + (2q_{\text{RO}} + q_{\text{dec}})\delta_{\text{dec}} + \frac{3q_{\text{RO}}}{|\mathcal{M}_{\text{FO}}|},$$

where $q_{\text{RO}} = q_{\mathcal{G}} + q_{\mathcal{H}}$.

Theorem 4 (Quantum ROM). *Assume that $\mathcal{G}$ and $\mathcal{H}$ are quantum-accessible random oracles, while decapsulation queries remain classical. For every quantum adversary $\mathcal{A}$ against the IND-CCA security of Frost.KEM making at most $q_{\mathcal{G}}$ queries to $\mathcal{G}_{\text{FO}}$, at most $q_{\mathcal{H}}$ queries to the domain-separated instances of $\mathcal{H}$, and at most q_{dec} decapsulation queries, there exists an IND-CPA adversary $\mathcal{B}$ against Frost.PKE such that*

$$\text{Adv}_{\text{Frost.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq \sqrt{\frac{2q_{\mathcal{H}}}{|\mathcal{M}_{\text{FO}}|}} + 4q_{\mathcal{G}}\sqrt{\delta_{\text{dec}}} + 2q_{\text{RO}}\sqrt{\text{Adv}_{\text{Frost.PKE}}^{\text{IND-CPA}}(\mathcal{B}) + \frac{1}{|\mathcal{M}_{\text{FO}}|}}.$$

For all profiles and variants considered in this document, $|\mathcal{M}_{\text{FO}}| = 2^{\kappa_{\text{m}}}$ with $\kappa_{\text{m}} \in \{128, 192, 256, 384, 512\}$. The proofs of Theorems 3 and 4 are given in Appendix B.4; they follow the standard FO and implicit-rejection KEM analyses in the ROM and QROM [FO99, HHK17, JZC⁺18].

6.4 Concrete Security

Table 10: Concrete security estimates for the default Frost profiles. Sizes are in bytes. Security estimates are in bits.

Profile	pk	ct	sk _{KEM}	ss	log ₂ δ _{dec}	Security		
						MATZOV	CORESVP-C	CORESVP-Q
Frost-128	5152	5192	6736	16	−169.44	131.85	107.75	98.58
Frost-192	9712	9760	11528	24	−199.84	197.65	175.27	161.13
Frost-256	16776	15552	19416	32	−266.61	257.08	236.36	216.55
Frost-384	25096	37736	29032	48	−440.52	393.79	376.83	345.16
Frost-512	36432	72944	41728	64	−581.12	512.97	498.68	456.47

The concrete estimates are computed with the Lattice Estimator from the LWE anchors in Theorem 2 [APS15, ACD⁺18]. For each profile, the reported value is the minimum over the public-key anchor, the ciphertext-side anchors, and the standard LWE attack families considered by the estimator [Alb17, GJ21, EJK20, BLL⁺22]. The MATZOV column is the main concrete estimator output. The CoreSVP columns give classical and quantum reduction-cost baselines [CN11]. The ideal security experiments treat $\mathbf{A}$, $\mathbf{D}_{\text{pk}}$, $\mathbf{D}_{\text{u}}$, and $\mathbf{D}_{\text{v}}$ as uniform; in the implementation these objects are generated from public seeds using domain-separated XOF calls.

The compact-ciphertext variants for the high-security profiles keep the same $n = m$, q , secret distributions, compression widths, and scalar error distributions as the corresponding default profiles. They transpose the two message dimensions, so the public-key and ciphertext-side LWE anchors are exchanged. Since the reported concrete security value is the minimum over these anchors and the standard estimator attack families, this exchange preserves the overall concrete-security estimate. Thus Frost-CC-384 and Frost-CC-512 have the same reported security estimates as Frost-384 and Frost-512, respectively, while changing the allocation between public-key and ciphertext bytes.

7 Comparison with Similar Lattice KEMs

This section compares Frost with representative lattice-based KEMs from both a design and an efficiency perspective. The main comparison points are FrodoKEM and Sclou⁺, since both are built

from ordinary unstructured LWE matrices rather than ring or module algebra [BCD⁺16, WZZ⁺24]. All three schemes therefore occupy the conservative plain-matrix design space. Their main difference lies in how the public transcript obtains its effective error, how bandwidth is reduced, and how the correctness margin is controlled.

FrodoKEM represents the sampled-noise LWE design line. It keeps the public transcript close to standard LWE and uses explicitly sampled error distributions. **Scloud⁺** keeps the unstructured-LWE setting but improves efficiency through ternary secrets and Barnes–Wall lattice coding. **Frost** follows a third route: it uses public dithered quantization to transmit split compressed public-key and ciphertext components, and after dither compensation the transcript has additive LWE normal forms with explicit input-independent quantization errors. Thus **Frost** aims to preserve the conservative plain-matrix security setting while moving the bandwidth-reduction mechanism into the hardness-compatible transcript structure.

Table 11 reports public-key, ciphertext, and secret-key sizes, whole-ciphertext decryption-failure estimates, and implementation performance. The **Frost** rows use the native AES128 matrix backend with the E_8 message codec. For 384- and 512-bit security, the table also reports **Frost-CC**, the compact-ciphertext variants of the same **Frost** profiles. These variants change only the public-key/ciphertext allocation through a transposed message layout; they do not change the underlying design rationale, LWQ-to-LWE security argument, or DFR estimates. **FrodoKEM** and **Scloud⁺** do not provide corresponding 384- or 512-bit parameter sets in the compared specifications.

Table 11: Comparison of **Frost** with representative LWE and LWR based KEMs. DFR denotes $\log_2 \delta_{\text{dec}}$ under the whole-ciphertext failure convention. Efficiency is reported in kCycles.

Scheme	Type	q	Size (B)			DFR	Ref kCycles			AVX2 kCycles		
			pk	ct	sk		KG	Enc	Dec	KG	Enc	Dec
128-bit security												
Frost-128	LWQ	2^{15}	5152	5192	6736	−169	878	586	668	401	569	666
FrodoKEM-640 [BCD+16]	LWE	2^{15}	9616	9752	19888	−139	1493	7583	7629	1123	1271	1193
Scloud ⁺ -128 [WZZ+24]	LWE	2^{12}	7200	5456	–	−134	1177	1215	1167	999	1062	1014
R5N1-1CCA-0d [BBF+19]	LWR	2^{12}	5740	5788	5772	–	992	446	556	470	770	857
CCALizard-128 [CKLS18]	LWE+LWR	2^{11}	524320	955	34304	−154	63042	50	76	41113	33	56
192-bit security												
Frost-192	LWQ	2^{16}	9712	9760	11528	−200	2428	1421	1596	999	1367	1551
FrodoKEM-976 [BCD+16]	LWE	2^{16}	15632	15792	31296	−200	2945	12823	12735	2290	2396	2255
Scloud ⁺ -192 [WZZ+24]	LWE	2^{12}	11136	10832	–	−201	2369	2463	2384	2008	2040	1925
R5N1-3CCA-0d [BBF+19]	LWR	2^{15}	9660	9716	9708	–	4505	1277	1693	902	1389	1677
256-bit security												
Frost-256	LWQ	2^{16}	16776	15552	19416	−267	5153	2929	3139	2010	2638	2873
FrodoKEM-1344 [BCD+16]	LWE	2^{16}	21520	21696	43088	−253	5358	29818	29988	4603	4551	3909
Scloud ⁺ -256 [WZZ+24]	LWE	2^{12}	18744	16916	–	−266	3826	4236	4225	2953	3248	3150
R5N1-5CCA-0d [BBF+19]	LWR	2^{15}	14636	14708	14700	–	6123	1467	1849	1421	2649	2256
384-bit security												
Frost-384	LWQ	2^{16}	25096	37736	29032	−441	11387	7406	7892	3915	6213	6650
Frost-CC-384	LWQ	2^{16}	37628	25204	43492	−441	13954	6723	7044	5378	6110	6455
512-bit security												
Frost-512	LWQ	2^{16}	36432	72944	41728	−581	21277	14673	15195	6805	12465	12902
Frost-CC-512	LWQ	2^{16}	72832	36544	83328	−581	26904	12028	12555	10582	10283	10789

7.1 Comparison with FrodoKEM

FrodoKEM is the closest conservative baseline for **Frost**. Both schemes use ordinary matrices over $\mathbb{Z}_q$, avoid ring and module algebra, and base their security analysis on plain LWE rather than on algebraically structured lattice assumptions. This common design choice gives a transparent security target and avoids known concerns that may arise from additional algebraic structure. It also imposes the same basic cost: dense matrix products and relatively large public data.

The main difference is how the transcript obtains its effective error. **FrodoKEM** follows the sampled-noise LWE route: the public key and ciphertext contain values derived from matrix products

plus explicitly sampled error terms. **Frost** instead uses public dithered quantization. The transmitted objects are split quantized values, and public dither compensation turns them into additive LWE normal forms with finite-support quantization errors. The quantization error is independent of the hidden quantization input, so compression remains compatible with the LWE security argument.

This design difference directly affects bandwidth. At 128-bit security, **Frost**-128 has a combined public-key and ciphertext footprint of 10344 bytes, compared with 19368 bytes for **FrodoKEM**-640. At 192- and 256-bit security, the corresponding combined footprints are 19472 and 32328 bytes for **Frost**, compared with 31424 and 43216 bytes for **FrodoKEM**. The performance numbers in Table 11 reflect the same effect: **Frost** still pays for unstructured matrix arithmetic, but reduced transcript sizes and a power-of-two implementation path give lower native cycle counts at the compared security levels.

7.2 Comparison with **Scloud**⁺

Scloud⁺ and **Frost** have a closer efficiency target. Both start from the premise that unstructured LWE is attractive for long-term conservatism, but that a direct **FrodoKEM**-like design carries substantial communication cost. Both schemes therefore add a bandwidth- and correctness-oriented layer on top of ordinary matrix arithmetic.

The mechanisms are different. **Scloud**⁺ uses ternary secrets and Barnes–Wall lattice coding to improve the decryption-noise tolerance and reduce parameter sizes [WZZ⁺24]. Its lattice code is a central component in the efficiency gain. **Frost** uses dithered quantization as the compression mechanism for the public transcript. Its E_8 layer is used for message encoding and correctness, while the hardness interface remains the LWQ transcript and the LWE normal form obtained after dither compensation.

This gives **Frost** a more explicit link between compression and the LWE analysis. The effective errors in the public key and ciphertext are not sampled separately and are not deterministic rounding errors tied to the hidden input. They are induced by public dithered quantization and have explicit finite-support distributions determined by the compression widths. The DFR analysis is then separated from the hardness argument and evaluated through the E_8 decoding region.

At 128-, 192-, and 256-bit security, the combined public-key and ciphertext footprints of **Frost** are 10344, 19472, and 32328 bytes. The corresponding values for **Scloud**⁺ are 12656, 21968, and 35660 bytes. Thus **Frost** gives smaller combined communication sizes at all three compared security levels. The native performance numbers are also competitive with **Scloud**⁺, with the difference mainly reflecting the different optimization routes: **Scloud**⁺ gains from ternary secrets and Barnes–Wall coding, whereas **Frost** gains from split quantized transmission, simple power-of-two arithmetic, and an E_8 -coded message layer with exact finite-support correctness analysis.

7.3 Other LWE/LWR-Based KEMs

Round5 and CCALizard provide useful context for bandwidth-oriented LWE/LWR designs, but they are not the primary baselines for **Frost**. Their hardness interfaces and design trade-offs differ from those of plain unstructured LWE KEMs. Round5 is based on LWR, while CCALizard combines LWE and LWR components [BBF⁺19, CKLS18]. These schemes show that deterministic rounding can be an effective way to reduce bandwidth, but deterministic rounding errors are tied to the hidden linear values.

The distinction is important for **Frost**. **Frost** uses a rounding-like compression channel, but public dithering restores an additive input-independent error distribution after compensation. This is the main conceptual separation from LWR-based transcripts. It lets **Frost** keep the bandwidth advantage of compression while retaining a direct LWE normal-form security route.

The 384- and 512-bit **Frost** profiles extend the same design principle beyond the security levels commonly provided by **FrodoKEM** and **Scloud**⁺. The corresponding **Frost**-CC rows are compact-ciphertext variants of the same profiles, not separate algorithms. They transpose the message dimensions and shift part of the communication cost from ciphertext to public key. This changes

the byte allocation and some implementation timings, but it preserves the same LWQ transcript structure, LWE anchors, concrete security estimates, and whole-ciphertext DFR bounds. Overall, the comparison indicates that **Frost** occupies a design point between conservative sampled-noise LWE and bandwidth-optimized LWR or hybrid schemes: it keeps the plain-matrix setting, uses quantization as a proof-compatible compression mechanism, and obtains the resulting bandwidth and performance improvements without relying on ring or module algebra.

8 Advantages and Limitations

8.1 Advantages

- **Simple power-of-two arithmetic.** Frost uses $q = 2^{15}$ for Frost-128 and $q = 2^{16}$ for the remaining profiles, as shown in Table 3. Coefficients fit naturally in a 16-bit integer path, and modular reduction can be implemented with fixed-width masks and shifts.
- **Efficient quantization and reconstruction.** The quantization and reconstruction routines use integer additions, shifts, masks, and fixed rounding rules [Zam14, LLL24a]. They do not require floating-point arithmetic, modular inversion, Gaussian sampling, or rejection sampling.
- **Uniform implementation structure.** All five profiles share the same algorithmic flow. Implementations only change compile-time constants such as $n, m, \ell_r, \ell_s, (t_{pk}, t_u, t_v), b_{msg}$, and (η_s, η_r) .
- **Conservative unstructured setting.** Frost uses ordinary matrices over $\mathbb{Z}_q$, without ring or module algebra. This keeps the underlying hard instances in the plain LWE setting and avoids algebraic structure in the public matrix [Reg05, BCD⁺16].
- **Explicit error distributions.** After public dither compensation, the transcript has the normal form. The distributions of $\mathbf{E}_{pk}$, $\mathbf{E}_u$, and $\mathbf{E}_v$ are determined directly by the compression parameters [LLL24a].
- **Finite-support correctness bound.** The E_8 -coded message layer is evaluated using finite-support component distributions and a root-projection Chernoff upper bound. The reported decryption-failure entries in Table ?? are whole-ciphertext bounds and are used directly in the KEM security analysis.

8.2 Limitations

- **Large bandwidth.** The unstructured design leads to larger public keys and ciphertexts than module- or ring-based KEMs. This is most visible in the high-security profiles: Frost-384 has a 25096-byte public key and a 37736-byte ciphertext, while Frost-512 has a 36432-byte public key and a 72944-byte ciphertext.
- **Dense matrix arithmetic.** The main computational cost comes from unstructured matrix products. AVX2 vectorization reduces this cost, but the arithmetic remains heavier than polynomial-ring or module-lattice arithmetic [BCD⁺16, BOR⁺21].
- **Bandwidth–correctness trade-off.** Increasing t_u or t_v improves the correctness margin by reducing quantization error, but increases ciphertext size. Increasing n and m raises the security margin, but also increases bandwidth and computation.
- **Limited suitability for constrained deployments.** The larger profiles may be unsuitable for low-bandwidth channels, small embedded devices, or applications with strict memory limits.

8.3 Side-Channel Considerations

- Implementations must avoid secret-dependent branches and secret-dependent memory accesses [KRR⁺18, HPRR20].

- The main leakage-sensitive operations are secret sampling, matrix products involving $\mathbf{S}$ or $\mathbf{R}$, message decoding, re-encryption during decapsulation, ciphertext comparison, and final shared-secret selection.
- Decapsulation must compute both candidate shared secrets and select the output with a constant-time mask. The comparison between ct and ct' must scan the full ciphertext.
- The public dithers $\mathbf{D}_{\text{pk}}$, $\mathbf{D}_{\text{u}}$, and $\mathbf{D}_{\text{v}}$ are generated from public seeds and do not require secret protection.
- Masking, shuffling, leakage testing, and platform-specific hardening are implementation tasks. They do not change the algorithmic specification or the LWE security estimates.

9 Implementation Specification

This section specifies the engineering conventions required for interoperable **Frost** implementations. The algorithms in the previous sections define the mathematical operations. This section fixes the corresponding byte layouts, integer representations, expansion rules, sampling order, quantization rules, message encoding, and constant-time decapsulation behavior.

The portable reference implementation is the implementation baseline. Optimized implementations may use different internal layouts, vectorized arithmetic, or precomputation strategies, but they must produce identical public keys, ciphertexts, shared secrets, and Known Answer Test outputs for the same inputs. All external objects are byte strings. Matrices are serialized by processing coefficients in row-major order unless stated otherwise.

9.1 External API and Implementation Boundary

For each submitted **Frost** instance, the implementation exposes the KEM interface

$$\text{crypto_kem_keypair}_\lambda(\text{pk}, \text{sk}), \quad \text{crypto_kem_enc}_\lambda(\text{ct}, \text{ss}, \text{pk}), \quad \text{crypto_kem_dec}_\lambda(\text{ss}, \text{ct}, \text{sk}).$$

The declared byte lengths are profile-dependent constants: $|\text{pk}| = \text{CRYPTO_PUBLICKEYBYTES}$, $|\text{sk}| = \text{CRYPTO_SECRETKEYBYTES}$, $|\text{ct}| = \text{CRYPTO_CIPHERTEXTBYTES}$, $|\text{ss}| = \text{CRYPTO_BYTES}$.

For the default profiles, these values are those reported in Table 3. For the compact-ciphertext variants, the public-key and ciphertext lengths are those reported in Table 4.

The implementation uses a common macro-instantiated core. Each instance fixes the compile-time constants $n, \ell_r, \ell_s, q, t_{\text{pk}}, t_{\text{u}}, t_{\text{v}}, \eta_s, \eta_r, b_{\text{msg}}$, and then instantiates the same KEM flow. The default profiles use $\ell_r = 8$, with $\ell_s = 8, 8, 8, 12, 16$ for security levels 128, 192, 256, 384, 512, respectively. The compact-ciphertext variants for the high-security profiles exchange ℓ_r and ℓ_s , giving $(\ell_r, \ell_s) = (12, 8)$ for **Frost-CC-384** and $(16, 8)$ for **Frost-CC-512**.

The `API_PKC` submission package is the conformance boundary for the reference and optimized x86 implementations. Native benchmark code may use different internal low-level backends, but it must preserve the same external byte strings and KEM semantics when instantiated with the same parameter set.

9.2 Integer Representation

All coefficients are represented internally as unsigned 16-bit words. The arithmetic modulus is $q = 2^{15}$ for **Frost-128** and $q = 2^{16}$ for the remaining profiles. Reduction modulo q is implemented by masking with $q - 1$. The centered interpretation is used only when required by decoding or analysis; serialized values use the canonical nonnegative representation.

For $x \in \{\text{pk}, \text{u}, \text{v}\}$, set $p_x = 2^{t_x}$, $\Delta_x = q/p_x$, and $g_x = \log_2 \Delta_x = \log_2 q - t_x$. The implementation uses shifts for multiplication and division by Δ_x .

9.3 Serialized Objects

The public key consists of a 32-byte public seed ρ , followed by the packed public-key matrix:

$$\text{pk} = \rho \parallel \text{Pack}_{t_{\text{pk}}}(\mathbf{B}), \quad |\text{pk}| = 32 + \left\lceil \frac{m\ell_r t_{\text{pk}}}{8} \right\rceil.$$

Here ρ determines the public matrix $\mathbf{A}$ and the public-key dither $\mathbf{D}_{\text{pk}}$.

At the algorithmic level, the ciphertext is the tuple $(\mu, \mathbf{U}, \mathbf{V})$, where μ is the public seed for the ciphertext-side dithers. The submitted byte string stores the same information in the order

$$\text{ct} = \text{Pack}_{t_u}(\mathbf{U}) \parallel \text{Pack}_{t_v}(\mathbf{V}) \parallel \text{salt},$$

where **salt** is the 32-byte serialization of μ . Thus

$$|\text{ct}| = 32 + \left\lceil \frac{n\ell_s t_u + \ell_r \ell_s t_v}{8} \right\rceil.$$

The order of the serialized ciphertext differs from the mathematical tuple only for packing convenience; the stored salt is still the public seed μ used to regenerate $(\mathbf{D}_u, \mathbf{D}_v)$.

The KEM secret key stores the packed PKE secret matrix, the public key, the public-key hash, and the implicit-rejection fallback string: $\text{sk}_{\text{KEM}} = \text{Pack}_{t_s}^{\text{sec}}(\mathbf{S}) \parallel \text{pk} \parallel h_{\text{pk}} \parallel z$, $t_s = \lceil \log_2(2\eta_s + 1) \rceil$. The packing $\text{Pack}_{t_s}^{\text{sec}}$ uses the fixed centered-secret encoding for entries of $\mathbf{S} \in \{-\eta_s, \dots, \eta_s\}^{n \times \ell_r}$. The public-key hash h_{pk} is a 32-byte string, while the fallback string z has length $\kappa_{\text{ss}}/8$ bytes in the submitted external secret-key layout. Consequently,

$$|\text{sk}_{\text{KEM}}| = |\text{pk}| + \left\lceil \frac{n\ell_r t_s}{8} \right\rceil + 32 + \frac{\kappa_{\text{ss}}}{8}.$$

These are the secret-key byte lengths reported in Tables 3 and 5.

Table 12: Serialized object layout in the submitted **Frost** implementation.

Object	Layout
pk	$\rho \parallel \text{Pack}_{t_{\text{pk}}}(\mathbf{B})$
ct	$\text{Pack}_{t_u}(\mathbf{U}) \parallel \text{Pack}_{t_v}(\mathbf{V}) \parallel \text{salt}$
sk _{KEM}	$\text{Pack}_{t_s}^{\text{sec}}(\mathbf{S}) \parallel \text{pk} \parallel h_{\text{pk}} \parallel z$
ss	$\kappa_{\text{ss}}/8$ -byte shared secret

9.4 Bit Packing and Parsing

For a vector of coefficients $(x_0, \dots, x_{L-1})$, Pack_t serializes the t least significant bits of each coefficient. The coefficients are processed in increasing index order. For each coefficient, the selected t -bit word is emitted from its most significant selected bit to its least significant selected bit. Output bytes are filled from bit position 7 down to bit position 0.

The inverse function Unpack_t reads the bit string in the same order and reconstructs coefficients in $\{0, \dots, 2^t - 1\}$. Unused output coefficients are set to zero before parsing. This convention applies uniformly to $\mathbf{B}$, $\mathbf{U}$, and $\mathbf{V}$.

9.5 Seed Expansion and Matrix Generation

The implementation uses deterministic expansion for public matrices, secret matrices, ephemeral matrices, public dithers, and key derivation. The profile-local XOF is SHAKE128 for **Frost-128** and SHAKE256 for the remaining profiles.

The public matrix $\mathbf{A}$ is expanded from the public seed ρ . The submitted code supports an AES128 matrix backend and a SHAKE matrix backend. The backend is an implementation choice,

but a selected backend must be used consistently for all operations and must produce the same matrix $\mathbf{A}$ from the same seed ρ . For the AES backend, input blocks contain little-endian row and column counters and are encrypted under ρ using AES128. For the SHAKE backend, each row is expanded from a two-byte little-endian row index followed by ρ .

The public dithers are expanded from their public seeds and component labels. For $\mathbf{D}_{\text{pk}}$, the seed is ρ . For $\mathbf{D}_{\text{u}}$ and $\mathbf{D}_{\text{v}}$, the seed is the ciphertext salt. For a component $x \in \{\text{pk}, \text{u}, \text{v}\}$, the expanded dither coefficients are parsed as little-endian 16-bit words and masked into $\{0, \dots, \Delta_x - 1\} = \{0, \dots, 2^{g_x} - 1\}$.

Table 13: Dithered quantization inputs in the submitted implementation.

Component	Width	Dither seed
B	t_{pk}	ρ
U	t_{u}	salt
V	t_{v}	salt

9.6 Secret Sampling

Secret and ephemeral matrices are sampled from centered binomial distributions by first expanding a seed into 16-bit words and then applying a deterministic bit-slicing map. For $\mathcal{B}_\eta$, each 16-bit word x is mapped to

$$\sum_{j=0}^{\eta-1} ((x \gg j) \& 1) - \sum_{j=0}^{\eta-1} ((x \gg (\eta + j)) \& 1).$$

The result is stored modulo q in the internal unsigned representation. The long-term matrix $\mathbf{S}$ uses η_s , while the ephemeral matrix $\mathbf{R}$ uses η_r . The entries are filled in increasing row-major order.

9.7 Dithered Quantization and Compensation

Let $x \in \{\text{pk}, \text{u}, \text{v}\}$, and let t_x , Δ_x , and $g_x = \log_2 \Delta_x$ be the corresponding compression width, quantization step, and shift value. For an input coefficient $a \in \mathbb{Z}_q$ and a dither coefficient $d \in \{0, \dots, \Delta_x - 1\}$, the transmitted label is the p_x -ary coefficient

$$b = \mathbf{Q}_{\Delta_x \mathbb{Z}, d}(a) / \Delta_x = ((a + d + 2^{g_x-1}) \gg g_x) \& (2^{t_x} - 1).$$

Equivalently, $b \in \mathbb{Z}_{p_x}$ is the compact coefficient stored by the implementation. The corresponding compensated normal-form coefficient is

$$\hat{a} = \Delta_x b - d = ((b \ll g_x) - d) \& (q - 1).$$

This rule is used for the public-key component, the **U** ciphertext component, and the **V** ciphertext component.

Thus the implementation stores split labels

$$\mathbf{B} \in \mathbb{Z}_{p_{\text{pk}}}^{m \times \ell_r}, \quad \mathbf{U} \in \mathbb{Z}_{p_{\text{u}}}^{n \times \ell_s}, \quad \mathbf{V} \in \mathbb{Z}_{p_{\text{v}}}^{\ell_r \times \ell_s}.$$

Encryption and decryption reconstruct the compensated forms

$$\hat{\mathbf{B}} = \Delta_{\text{pk}} \mathbf{B} - \mathbf{D}_{\text{pk}}, \quad \hat{\mathbf{U}} = \Delta_{\text{u}} \mathbf{U} - \mathbf{D}_{\text{u}}, \quad \hat{\mathbf{V}} = \Delta_{\text{v}} \mathbf{V} - \mathbf{D}_{\text{v}}$$

coefficientwise modulo q .

Table 14: Message embedding layout for the submitted **Frost** profiles.

Profile	$ M $	b_{msg}	Bits/block	N_{blk}
Frost-128	16 bytes	2	16	8
Frost-192	24 bytes	3	24	8
Frost-256	32 bytes	4	32	8
Frost-384	48 bytes	4	32	12
Frost-CC-384	48 bytes	4	32	12
Frost-512	64 bytes	4	32	16
Frost-CC-512	64 bytes	4	32	16

9.8 Message Encoding and Decoding

All submitted profiles use the E_8 message codec. The FO plaintext has length $\kappa_m = b_{\text{msg}} \ell_r \ell_s$ bits and is stored as a byte string of length $\kappa_m/8$. It is parsed into $N_{\text{blk}} = \ell_r \ell_s/8$ consecutive 8-dimensional message blocks. Each block carries $8b_{\text{msg}}$ message bits, and all blocks are active.

Let $\alpha_{E_8} = q/2^{b_{\text{msg}}}$ and $\mathcal{C}_{E_8} = \alpha_{E_8} E_8 \cap [0, q)^8$. Within one block, the next $8b_{\text{msg}}$ message bits are parsed into eight small integers using the coordinate-width vector

$$\mathbf{b} = (b_{\text{msg}} - 1, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}}, b_{\text{msg}} + 1).$$

These widths sum to $8b_{\text{msg}}$. The input bits are read little-endian within each byte and are consumed consecutively from the FO plaintext byte string. For block j , let the parsed integers be $(a_{j,0}, a_{j,1}, \dots, a_{j,7})$, where $0 \leq a_{j,h} < 2^{b_h}$. The label and delabel operations use the standard rectangular-form labeling rule specialized to E_8 . We use the following row-basis convention: $E_8 = \{\mathbf{zG}_{E_8} : \mathbf{z} \in \mathbb{Z}^8\}$, where

$$\mathbf{G}_{E_8} = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \end{pmatrix}.$$

Set $P = 2^{b_{\text{msg}}}$, and define the mixed-radix vector

$$\mathbf{p}_{E_8} = (p_0, \dots, p_7) = (2^{b_0}, 2^{b_1}, \dots, 2^{b_7}).$$

By the choice of $\mathbf{b}$, the rows of $\text{diag}(\mathbf{p}_{E_8})\mathbf{G}_{E_8}$ generate the coarse lattice $P\mathbb{Z}^8$. Thus

$$\mathcal{I}_{E_8, \mathbf{b}} = \prod_{h=0}^7 \{0, \dots, p_h - 1\}$$

is a complete set of representatives for $E_8/P\mathbb{Z}^8$. Since $q = \alpha_{E_8}P$, this gives a public bijection between one message block and $\mathcal{C}_{E_8}$.

Algorithm 9 $\text{Label}_{E_8, \mathbf{b}}(a_0, \dots, a_7)$

Require: Integers $a_h \in \{0, \dots, p_h - 1\}$, $h = 0, \dots, 7$.

- 1: $\mathbf{a} \leftarrow (a_0, \dots, a_7)$.
 - 2: $\mathbf{y} \leftarrow \mathbf{aG}_{E_8} \in E_8$.
 - 3: $\mathbf{c} \leftarrow \alpha_{E_8} \mathbf{y} \bmod q$, reduced coordinatewise to $[0, q)^8$.
 - 4: **return** $\mathbf{c} \in \mathcal{C}_{E_8}$.
-

Algorithm 10 $\text{Delabel}_{E_8, \mathbf{b}}(\mathbf{c})$

Require: Codeword $\mathbf{c} \in \mathcal{C}_{E_8}$.

- 1: $\mathbf{y} \leftarrow \mathbf{c}/\alpha_{E_8}$, interpreted as an element of $E_8/P\mathbb{Z}^8$.
 - 2: $\tilde{\mathbf{a}} \leftarrow \mathbf{y}\mathbf{G}_{E_8}^{-1}$.
 - 3: **for** $h = 0, \dots, 7$ **do**
 - 4: $a_h \leftarrow \tilde{a}_h \bmod p_h$, reduced to $\{0, \dots, p_h - 1\}$.
 - 5: **end for**
 - 6: **return** $(a_0, \dots, a_7)$.
-

The default profiles have $\ell_r = 8$. For these profiles, the j -th message block is embedded into the j -th column of the message matrix: $\text{Encode}(M)_{*,j} = \mathbf{c}_j$, $j = 0, \dots, \ell_s - 1$. The compact-ciphertext variants for the high-security profiles have $\ell_s = 8$. For these variants, the i -th message block is embedded into the i -th row: $\text{Encode}(M)_{i,*} = \mathbf{c}_i$, $i = 0, \dots, \ell_r - 1$. For security levels 128, 192, and 256, the compact-ciphertext instance names have $(\ell_r, \ell_s) = (8, 8)$ and use the same placement as the corresponding default profiles.

Decoding first reconstructs the noisy message matrix

$$\mathbf{W} = \hat{\mathbf{V}} - \mathbf{S}^\top \hat{\mathbf{U}} \pmod{q}.$$

For the default profiles, the decoder applies E_8 nearest-neighbor decoding to each column $\mathbf{W}_{*,j}$, $j = 0, \dots, \ell_s - 1$. For the high-security compact-ciphertext variants, it applies the same decoder to each row $\mathbf{W}_{i,*}$, $i = 0, \dots, \ell_r - 1$. The decoded lattice point is mapped through $\text{Delabel}_{E_8, \mathbf{b}}$, and the recovered integers are serialized back to the FO plaintext in the same block order and little-endian bit order used during encoding.

The nearest-neighbor quantizer for $\alpha_{E_8} E_8$ uses the fixed tie convention specified for the lattice decoder. The recovered message is used only inside the FO re-encryption check. A decoding error does not produce an externally visible failure symbol; it leads to a re-encryption mismatch and is then handled by the KEM implicit-rejection rule.

9.9 Validation Requirements

A conforming implementation must satisfy the following requirements. The serialized **pk**, **ct**, **sk**, and **ss** byte strings must have exactly the declared instance lengths. Packing and unpacking must be bit-compatible with the reference implementation. Seed expansion, CBD sampling, dither generation, quantization, compensation, message encoding, and message decoding must be deterministic and match the reference output for the same inputs. Decapsulation must implement implicit rejection by constant-time comparison and constant-time selection. Optimized implementations must pass the same Known Answer Tests as the portable reference implementation and must preserve the same public API and byte layout.

A Mathematical Foundations

This section presents the hardness assumptions underlying Frost.

Definition 1 (LWE, [Reg05]). Let $n, m, q \in \mathbb{N}$, and let χ_s, χ_e be distributions over $\mathbb{Z}_q$. The LWE distribution $\text{LWE}_{q,n,m,\chi_s,\chi_e}$ is defined by $\mathbf{A} \leftarrow \mathcal{U}(\mathbb{Z}_q^{m \times n})$, $\mathbf{s} \leftarrow \chi_s^n$, $\mathbf{e} \leftarrow \chi_e^m$, and $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q}$. The output is $(\mathbf{A}, \mathbf{b})$. Computational problems on LWE are defined as follows.

1. **Search-LWE.** Given samples from $\text{LWE}_{q,n,m,\chi_s,\chi_e}$, recover $\mathbf{s}$.
2. **Decision-LWE.** Distinguish $\text{LWE}_{q,n,m,\chi_s,\chi_e}$ from the uniform distribution over $\mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$.

Definition 2 (LWQ, [LLL24a]). Let $n, m, q, p \in \mathbb{N}$ with $p \mid q$, let $\Delta = q/p$, let $\Lambda = \Delta\mathbb{Z}^m$, and let $\mathcal{S} = \{0, \dots, \Delta - 1\}^m$. The LWQ distribution $\text{LWQ}_{q,n,m,p,\chi_s}$ is defined by $\mathbf{A} \leftarrow \mathcal{U}(\mathbb{Z}_q^{m \times n})$, $\mathbf{s} \leftarrow \chi_s^n$, $\mathbf{d} \leftarrow \mathcal{U}(\mathcal{S})$, and $\mathbf{b} = \mathbf{Q}_\Lambda(\mathbf{A}\mathbf{s} + \mathbf{d}) - \mathbf{d} \pmod{q}$. The output is $(\mathbf{A}, \mathbf{b})$. Computational problems on LWQ are defined as follows.

1. **Search-LWQ.** Given samples from $\text{LWQ}_{q,n,m,p,\chi_s}$, recover $\mathbf{s}$.
2. **Decision-LWQ.** Distinguish $\text{LWQ}_{q,n,m,p,\chi_s}$ from the uniform distribution over $\mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$.

Definition 3 (Split-LWQ, [LLL24a]). Let $n, m, q, p \in \mathbb{N}$ with $p \mid q$, let $\Delta = q/p$, let $\Lambda = \Delta\mathbb{Z}^m$, and let $\mathcal{S} = \{0, \dots, \Delta - 1\}^m$. The Split-LWQ distribution $\text{SLWQ}_{q,n,m,p,\chi_s}$ is defined by $\mathbf{A} \leftarrow \mathcal{U}(\mathbb{Z}_q^{m \times n})$, $\mathbf{s} \leftarrow \chi_s^n$, $\mathbf{d} \leftarrow \mathcal{U}(\mathcal{S})$, and $\mathbf{c} = \mathbf{Q}_\Lambda(\mathbf{A}\mathbf{s} + \mathbf{d}) \pmod{q}$. The output is $(\mathbf{A}, \mathbf{c}, \mathbf{d})$. Computational problems on Split-LWQ are defined as follows.

1. **Search Split-LWQ.** Given samples from $\text{SLWQ}_{q,n,m,p,\chi_s}$, recover $\mathbf{s}$.
2. **Decision Split-LWQ.** Distinguish $\text{SLWQ}_{q,n,m,p,\chi_s}$ from the uniform distribution over $\mathbb{Z}_q^{m \times n} \times (\Lambda \cap \mathbb{Z}_q^m) \times \mathcal{S}$.

Note that protocol constructions typically use the Split-LWQ variant, as the dither is public. The standard LWQ form, by contrast, is reserved for security analysis.

Let $\chi_{q,p}$ denote the scalar quantization-error distribution induced by uniform dither over $\mathcal{S}$. For the power-of-two parameter sets used by Frost, if $\Delta = q/p$, then $\chi_{q,p}$ is uniform over $\{-\Delta/2 + 1, \dots, \Delta/2\}$.

Theorem 5. Let $p \mid q$, $\Delta = q/p$, $\Lambda = \Delta\mathbb{Z}^m$, and $\mathcal{S} = \{0, \dots, \Delta - 1\}$. For every fixed $\mathbf{x} \in \mathbb{Z}_q^m$, if $\mathbf{d} \leftarrow \mathcal{U}(\mathcal{S}^m)$, then the distribution of $\mathbf{e}_Q = \mathbf{Q}_\Lambda(\mathbf{x} + \mathbf{d}) - (\mathbf{x} + \mathbf{d})$ is independent of $\mathbf{x}$, and its coordinates are distributed according to $\chi_{q,p}$.

Proof. The quantizer onto $\Delta\mathbb{Z}^m$ is invariant under shifts by $\Delta\mathbb{Z}^m$. Hence the error map $E(\mathbf{z}) = \mathbf{Q}_\Lambda(\mathbf{z}) - \mathbf{z}$ depends only on $\mathbf{z} \pmod{\Delta}$. Since $\mathbf{d}$ is uniform over one complete residue system modulo Δ , the residue $\mathbf{x} + \mathbf{d} \pmod{\Delta}$ is uniform and independent of $\mathbf{x}$. Therefore $E(\mathbf{x} + \mathbf{d})$ has the same distribution for every $\mathbf{x}$, and the scalar distribution is precisely $\chi_{q,p}$. $\square$

Theorem 6. Let $p \mid q$, $\Delta = q/p$, and $\Lambda = \Delta\mathbb{Z}^m$. Then the real LWQ normal-form distribution $\text{LWQ}_{q,n,m,p,\chi_s}^{\text{real}}$ is identical to the real LWE distribution with error distribution $\chi_{q,p}$. The corresponding random distributions are also identical. Moreover, the Split-LWQ and LWQ normal-form decision experiments are related by a public coordinate-wise bijection. Consequently, for every distinguisher $\mathcal{A}$ against Split-LWQ, there exists a distinguisher $\mathcal{B}$ against LWE with the same distinguishing advantage, and conversely up to this public bijection.

Proof. For a real LWQ sample, set $\mathbf{x} = \mathbf{A}\mathbf{s}$. By Theorem 5, the error $\mathbf{e}_Q = \mathbf{Q}_\Lambda(\mathbf{A}\mathbf{s} + \mathbf{d}) - (\mathbf{A}\mathbf{s} + \mathbf{d})$ is independent of $(\mathbf{A}, \mathbf{s})$ and follows $\chi_{q,p}^m$. Thus $\mathbf{Q}_\Lambda(\mathbf{A}\mathbf{s} + \mathbf{d}) - \mathbf{d} = \mathbf{A}\mathbf{s} + \mathbf{e}_Q$, which is exactly the real LWE branch with error distribution $\chi_{q,p}$. For the random branch, adding an independent quantization error to a uniform element of $\mathbb{Z}_q^m$ preserves uniformity, so the LWQ random branch is the uniform LWE branch.

It remains to compare the split and normal views. Define $\Phi : (\Lambda \cap \mathbb{Z}_q^m) \times \mathcal{S}^m \rightarrow \mathbb{Z}_q^m$ by $\Phi(\mathbf{c}, \mathbf{d}) = \mathbf{c} - \mathbf{d} \pmod{q}$. Since $\Lambda = \Delta \mathbb{Z}^m$ and $q = p\Delta$, every scalar element of $\Lambda \cap \mathbb{Z}_q$ has the form Δa with $a \in \mathbb{Z}_p$. For every $y \in \mathbb{Z}_q$, the equation $y = \Delta a - d \pmod{q}$ has a unique solution with $a \in \mathbb{Z}_p$ and $d \in \{0, \dots, \Delta - 1\}$, namely $d \equiv -y \pmod{\Delta}$ and $a = (y + d)/\Delta \pmod{p}$. Therefore Φ is a coordinate-wise bijection. It maps real split samples to real normal-form samples and random split samples to random normal-form samples. The two decision experiments therefore have identical distinguishing advantage. $\square$

In **Frost**, the transmitted matrices $\mathbf{B}, \mathbf{U}, \mathbf{V}$ are compact split components, while the proof uses their compensated normal forms. For instance, $\hat{\mathbf{B}} = \Delta_{\mathbf{pk}} \mathbf{B} - \mathbf{D}_{\mathbf{pk}} = \mathbf{A}\mathbf{S} + \mathbf{E}_{\mathbf{pk}}$, and analogously $\hat{\mathbf{U}} = \Delta_{\mathbf{u}} \mathbf{U} - \mathbf{D}_{\mathbf{u}}$ and $\hat{\mathbf{V}} = \Delta_{\mathbf{v}} \mathbf{V} - \mathbf{D}_{\mathbf{v}}$. This is the bridge between the transmitted compact representation and the LWE anchors used in Section 6.

B Correctness and Security Details

This appendix gives the technical details behind Section 6. It contains the detailed decryption-failure computation, the proof of the LWQ-to-LWE correspondence, the IND-CPA game-hopping proof, and the ROM/QROM proofs for the Fujisaki–Okamoto transform.

B.1 Lattice Code and DFR Analysis

This subsection details the construction of the lattice-code layer and the methodology for computing the DFR. We employ 8-dimensional E_8 lattice codes for message encoding. Let $\mathcal{C}_{E_8} \subset \alpha_{E_8} E_8$ denote the codebook. A message block is mapped to a lattice point $\mathbf{c} \in \mathcal{C}_{E_8}$. Upon decryption, the received vector is corrupted by an error vector $\mathbf{e}$. Decoding succeeds if the error vector $\mathbf{e}$ resides within the Voronoi cell of $\mathbf{c}$. For the E_8 root system $\mathcal{R}(E_8)$, this condition is equivalent to:

$$|\langle \mathbf{e}, \mathbf{r} \rangle| < \alpha_{E_8} \quad \forall \mathbf{r} \in \mathcal{R}(E_8).$$

The decryption error is $\mathbf{E}_{\text{dec}} = \mathbf{E}_{\mathbf{pk}}^T \mathbf{R} - \mathbf{S}^T \mathbf{E}_{\mathbf{u}} + \mathbf{E}_{\mathbf{v}}$. The analysis addresses potential statistical dependencies across blocks as follows:

- **Underlying Independence:** Under the independent public-dither model, the component matrices $\mathbf{E}_{\mathbf{pk}}, \mathbf{E}_{\mathbf{u}}$, and $\mathbf{E}_{\mathbf{v}}$ are constructed to be statistically independent, both internally and with respect to their corresponding quantization inputs. This ensures that the scalar marginal distributions are well-defined, thereby enabling the convolution-based analysis of the error term.
- **Bounding Correlated Events:** Let $\mathcal{F}_k$ be the failure event for block $k \in \{1, \dots, N_{\text{blk}}\}$. The global failure event is $\mathcal{F} = \bigcup_{k=1}^{N_{\text{blk}}} \mathcal{F}_k$. By the subadditivity of probability measures:

$$\Pr[\mathcal{F}] \leq \sum_{k=1}^{N_{\text{blk}}} \Pr[\mathcal{F}_k].$$

This union bound holds strictly regardless of the statistical dependencies between blocks induced by the shared secret matrices $\mathbf{S}$ and $\mathbf{R}$. Thus, analyzing each block k individually via its projection $X_{\mathbf{r}}^{(k)} = \langle \mathbf{e}_k, \mathbf{r} \rangle$ provides a valid upper bound for δ_{dec} .

To estimate $\Pr[\mathcal{F}_k]$, we define the CGF $K_{\mathbf{r}}(\theta) = \log \mathbb{E}[\exp(\theta X_{\mathbf{r}})]$. Applying the Chernoff bound, for any $\theta > 0$:

$$\Pr[X_{\mathbf{r}} \geq \alpha_{E_8}] = \Pr[e^{\theta X_{\mathbf{r}}} \geq e^{\theta \alpha_{E_8}}] \leq e^{-\theta \alpha_{E_8}} \mathbb{E}[e^{\theta X_{\mathbf{r}}}] = \exp(K_{\mathbf{r}}(\theta) - \theta \alpha_{E_8}).$$

Minimizing over $\theta > 0$ yields the tightest upper bound. A symmetric argument applies to the lower tail with $\theta < 0$:

$$\Pr[X_{\mathbf{r}} \leq -\alpha_{E_8}] \leq \inf_{\theta < 0} \exp(K_{\mathbf{r}}(\theta) + \theta \alpha_{E_8}).$$

This approach circumvents the explicit convolution of the $m + n + 1$ error sources, as the CGF of the sum of independent components is the sum of their individual CGFs.

Combining the union bound over all message blocks and root projections, we define the overall DFR bound:

Proposition 1. *Let N_{blk} be the number of message-decoding blocks. The total decryption failure probability δ_{dec} satisfies:*

$$\delta_{\text{dec}} \leq N_{\text{blk}} \sum_{\mathbf{r} \in \mathcal{R}(E_8)} \left(\inf_{\theta > 0} \exp(K_{\mathbf{r}}(\theta) - \theta \alpha_{E_8}) + \inf_{\theta < 0} \exp(K_{\mathbf{r}}(\theta) + \theta \alpha_{E_8}) \right).$$

The security bounds reported in Table 10 are derived from this proposition, with the CGFs computed directly from the finite-support probability mass functions of the individual error components. This formulation yields a rigorous bound that captures cross-block correlations while remaining computationally tractable.

B.2 Proof of the LWQ-to-LWE Correspondence

Proof of Theorem 1. Let $p \mid q$, $\Delta = q/p$, and $\Lambda = \Delta \mathbb{Z}^N$. For a fixed input $\mathbf{x} \in \mathbb{Z}_q^N$, sample $\mathbf{d} \leftarrow \mathcal{U}(\{0, \dots, \Delta - 1\}^N)$, and define $\mathbf{e}_Q = Q_{\Lambda}(\mathbf{x} + \mathbf{d}) - (\mathbf{x} + \mathbf{d})$. Nearest-neighbor quantization onto $\Delta \mathbb{Z}^N$ is invariant under shifts by $\Delta \mathbb{Z}^N$. Hence $\mathbf{e}_Q$ depends only on $\mathbf{x} + \mathbf{d} \bmod \Delta$. Since $\mathbf{d}$ is uniform over one complete residue system modulo Δ , this residue is uniform and independent of $\mathbf{x}$. Therefore $\mathbf{e}_Q$ is independent of $\mathbf{x}$, and each coordinate follows $\chi_{q,p}$.

For a real LWQ normal-form sample, set $\mathbf{x} = \mathbf{A}\mathbf{s}$. Then $Q_{\Lambda}(\mathbf{A}\mathbf{s} + \mathbf{d}) - \mathbf{d} = \mathbf{A}\mathbf{s} + \mathbf{e}_Q$, where $\mathbf{e}_Q \leftarrow \chi_{q,p}^N$ is independent of $(\mathbf{A}, \mathbf{s})$. This is exactly the real LWE distribution with secret distribution χ_s and error distribution $\chi_{q,p}$. The random normal-form LWQ distribution is uniform over the same ambient space, which is exactly the random branch of decisional LWE.

It remains to relate the split representation to the normal representation. For one coordinate $b \in \mathbb{Z}_q$, there is a unique $d \in \{0, \dots, \Delta - 1\}$ satisfying $d \equiv -b \pmod{\Delta}$, and then $a = (b + d)/\Delta \pmod{p}$ is the unique element of $\mathbb{Z}_p$ satisfying $b = \Delta a - d \pmod{q}$. Thus the map $(a, d) \mapsto \Delta a - d \pmod{q}$ is a bijection from $\mathbb{Z}_p \times \{0, \dots, \Delta - 1\}$ to $\mathbb{Z}_q$. Applied coordinatewise, it maps real split samples to real normal-form samples and uniform split samples to uniform normal-form samples. Therefore split-form and normal-form decision experiments have the same distinguishing advantage. $\square$

B.3 Proof of IND-CPA Security

The IND-CPA proof uses the game sequence in Figure 1. The proof is written after replacing the concrete seed-expansion procedures by ideal expansion procedures. The cost of this replacement is the term $\text{Adv}_{\text{Exp}}(\mathcal{C}_{\text{Exp}})$ in Theorem 2.

Proof of Theorem 2. The transition from the seeded experiment to G_0 replaces the concrete expansion procedures by the ideal expansion experiment, which costs $\text{Adv}_{\text{Exp}}(\mathcal{C}_{\text{Exp}})$. The transition from G_0 to G_1 replaces $\mathbf{B}$ by a uniform matrix in $\mathbb{Z}_{p_{\text{pk}}}^{m \times \ell_r}$. In G_0 , the reconstructed public-key value is $\hat{\mathbf{B}} = \mathbf{A}\mathbf{S} + \mathbf{E}_{\text{pk}} \pmod{q}$, with $\mathbf{S} \leftarrow \mathcal{B}_{\eta_s}^{n \times \ell_r}$ and $\mathbf{E}_{\text{pk}} \leftarrow \chi_{q,p_{\text{pk}}}^{m \times \ell_r}$. By Theorem 1, this hop is bounded by the public-key LWE advantage in Theorem 2.

The transition from G_1 to G_2 replaces $\mathbf{U}$ and $\mathbf{V}$ jointly by uniform compact matrices. In G_1 , the public-key normal form is independent of $\mathbf{S}$, and the stacked ciphertext normal form is

$$\begin{pmatrix} \hat{\mathbf{U}} \\ \hat{\mathbf{V}} - \text{Encode}(M_b) \end{pmatrix} = \begin{pmatrix} \mathbf{A}^\top \\ \hat{\mathbf{B}}^\top \end{pmatrix} \mathbf{R} + \begin{pmatrix} \mathbf{E}_u \\ \mathbf{E}_v \end{pmatrix}.$$

The upper n error rows follow χ_{q,p_u} , and the last ℓ_r error rows follow χ_{q,p_v} . The shift by $\text{Encode}(M_b)$ is fixed after the challenge message is chosen and preserves both branches of the decisional experiment.

Game G_0 (Real)	Game G_1 (Random PK)	Game G_2 (Random CT)
Setup 1. $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$ 2. $(\mathbf{A}, \mathbf{D}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$ 3. $\mathbf{S} \leftarrow \mathcal{B}_{\eta_s}^{n \times \ell_r}$ 4. $\mathbf{B} \leftarrow \mathbf{Q}_{\Lambda_{\text{pk}}, \mathbf{D}_{\text{pk}}}(\mathbf{AS})/\Delta_{\text{pk}}$	Setup 1. $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$ 2. $(\mathbf{A}, \mathbf{D}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$ 3. $\mathbf{B} \leftarrow \mathcal{U}(\mathbb{Z}_{p_{\text{pk}}}^{m \times \ell_r})$	Setup 1. $\rho \leftarrow \mathcal{U}(\{0, 1\}^{256})$ 2. $(\mathbf{A}, \mathbf{D}_{\text{pk}}) \leftarrow \text{GenPublic}(\rho)$ 3. $\mathbf{B} \leftarrow \mathcal{U}(\mathbb{Z}_{p_{\text{pk}}}^{m \times \ell_r})$
Challenge (M_b) 5. $\mathbf{R} \leftarrow \mathcal{B}_{\eta_r}^{m \times \ell_s}$ 6. $\mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$ 7. $(\mathbf{D}_u, \mathbf{D}_v) \leftarrow \text{GenDither}(\mu)$ 8. $\hat{\mathbf{B}} \leftarrow \Delta_{\text{pk}} \mathbf{B} - \mathbf{D}_{\text{pk}} \pmod{q}$ 9. $\mathbf{U} \leftarrow \mathbf{Q}_{\Lambda_u, \mathbf{D}_u}(\mathbf{A}^T \mathbf{R})/\Delta_u$ 10. $\mathbf{V} \leftarrow \mathbf{Q}_{\Lambda_v, \mathbf{D}_v}(\hat{\mathbf{B}}^T \mathbf{R} + \text{Encode}(M_b))/\Delta_v$	Challenge (M_b) 4. $\mathbf{R} \leftarrow \mathcal{B}_{\eta_r}^{m \times \ell_s}$ 5. $\mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$ 6. $(\mathbf{D}_u, \mathbf{D}_v) \leftarrow \text{GenDither}(\mu)$ 7. $\hat{\mathbf{B}} \leftarrow \Delta_{\text{pk}} \mathbf{B} - \mathbf{D}_{\text{pk}} \pmod{q}$ 8. $\mathbf{U} \leftarrow \mathbf{Q}_{\Lambda_u, \mathbf{D}_u}(\mathbf{A}^T \mathbf{R})/\Delta_u$ 9. $\mathbf{V} \leftarrow \mathbf{Q}_{\Lambda_v, \mathbf{D}_v}(\hat{\mathbf{B}}^T \mathbf{R} + \text{Encode}(M_b))/\Delta_v$	Challenge (M_b) 4. $\mu \leftarrow \mathcal{U}(\{0, 1\}^{256})$ 5. $(\mathbf{D}_u, \mathbf{D}_v) \leftarrow \text{GenDither}(\mu)$ 6. $\mathbf{U} \leftarrow \mathcal{U}(\mathbb{Z}_{p_u}^{n \times \ell_s})$ 7. $\mathbf{V} \leftarrow \mathcal{U}(\mathbb{Z}_{p_v}^{\ell_r \times \ell_s})$

Figure 1: Game sequence for the IND-CPA proof of Frost.PKE. The first hop replaces the public-key split component by a uniform compact matrix. The second hop replaces the ciphertext split components jointly, since both are generated from the same ephemeral secret $\mathbf{R}$.

By Theorem 1, the second hop is bounded by the row-wise mixed-error LWE advantage in Theorem 2. In G_2 , both ciphertext components are uniform and independent of the challenge bit, so the adversary succeeds with probability $1/2$. Summing the expansion replacement and the two LWE hops gives the theorem. $\square$

B.4 Proof of IND-CCA Security

Proof of Theorem 3. We use the concrete IND-CPA bound for the message-based implicit-rejection transform KEM_m^ℓ of Hofheinz, Hövelmanns, and Kiltz [HHK17]. For this transform, the classical ROM loss is

$$(2q_{\text{RO}} + q_{\text{dec}})\delta + \frac{3q_{\text{RO}}}{|\mathcal{M}|} + 3\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}.$$

The scheme-specific condition is the deterministic re-encryption property of the underlying PKE once the public key and FO message are fixed. In Frost, encapsulation derives

$$(r_{\text{Enc}}, K') \leftarrow \mathcal{G}_{\text{FO}}(h_{\text{pk}} \parallel M),$$

and parses r_{Enc} as (σ, μ) . The seed σ fixes the ephemeral secret matrix $\mathbf{R}$, and the seed μ fixes the ciphertext-side dithers $(\mathbf{D}_u, \mathbf{D}_v)$. The seed ρ contained in the public key fixes $\mathbf{A}$ and the public-key dither $\mathbf{D}_{\text{pk}}$. Thus, once (pk, M) is fixed, $\text{Frost.PKE.Enc}(\text{pk}, M, r_{\text{Enc}})$ is deterministic.

The decapsulation algorithm decrypts, re-derives $(r'_{\text{Enc}}, K') \leftarrow \mathcal{G}_{\text{FO}}(h_{\text{pk}} \parallel M')$, re-encrypts using r'_{Enc} , compares the full ciphertext, and applies implicit rejection. The ciphertext comparison includes the split ciphertext components and the public seed μ carried by the ciphertext. Thus Frost.KEM has the required message-based implicit-rejection FO form. Substituting $\mathcal{M} = \mathcal{M}_{\text{FO}}$, $\delta = \delta_{\text{dec}}$, and $q_{\text{RO}} = q_{\mathcal{G}} + q_{\mathcal{H}}$ gives the stated inequality. $\square$

Proof of Theorem 4. We apply the implicit-rejection FO KEM bound in the quantum random-oracle model of Jiang, Zhang, Chen, Wang, and Ma [JZC⁺18, JZM19]. For a quantum adversary making $q_{\mathcal{G}}$ queries to $\mathcal{G}_{\text{FO}}$, $q_{\mathcal{H}}$ queries to the domain-separated instances of $\mathcal{H}$, and classical decapsulation queries, the bound contains the message-search term, the square-root amplification of the decryption-failure event, and a square-root loss in the underlying one-way PKE advantage.

For uniformly sampled messages over $\mathcal{M}_{\text{FO}}$, IND-CPA security implies

$$\text{Adv}_{\text{Frost.PKE}}^{\text{OW-CPA}} \leq \text{Adv}_{\text{Frost.PKE}}^{\text{IND-CPA}} + \frac{1}{|\mathcal{M}_{\text{FO}}|}.$$

Indeed, an adversary that recovers the encrypted message with probability p gives a distinguisher between an encryption of that message and an encryption of an independent random message with advantage at least $p - 1/|\mathcal{M}_{\text{FO}}|$.

The deterministic re-encryption property follows from the same FO coin derivation as in the classical proof. Once $(\mathbf{pk}, M)$ is fixed, the value r_{Enc} is determined by $\mathcal{G}_{\text{FO}}(h_{\mathbf{pk}} \parallel M)$. Parsing r_{Enc} as (σ, μ) fixes both the ephemeral secret matrix and the ciphertext-side dithers, while the public key fixes $\mathbf{A}$ and $\mathbf{D}_{\mathbf{pk}}$. The only correctness loss is the event that decryption crosses the decoding boundary, which is accounted for by δ_{dec} . Substituting $\mathcal{M} = \mathcal{M}_{\text{FO}}$, $\delta = \delta_{\text{dec}}$, and $q_{\text{RO}} = q_{\mathcal{G}} + q_{\mathcal{H}}$ gives the stated inequality. $\square$

C Native x86 Implementation Benchmarks

This appendix reports the native x86 implementation benchmark. Unlike the API_PKC self-evaluation results in Section 5.2, these measurements use the implementation’s native low-level backend and are intended to show the engineering performance of the underlying arithmetic implementation. They are not a replacement for the API_PKC conformance-oriented self-evaluation results.

Unless stated otherwise, the native benchmark uses the AES128 matrix backend and the E_8 message codec. Cycle counts are reported in kCycles and rounded to the nearest kCycle. The native x86 benchmark uses the x86-64 platform summarized in Table 6.

Table 15 reports the native full-KEM results for the default **Frost** instances and the two nontrivial compact-ciphertext instances. For security levels 128, 192, and 256, the compact-ciphertext instance names have the same external public-key and ciphertext lengths as the corresponding default instances, so they are not repeated in the table. For security levels 384 and 512, the compact-ciphertext instances use the layout described in Table 4.

Table 15: Native x86 implementation efficiency of **Frost** in kCycles.

Instance	Reference			AVX2		
	KeyGen	Encaps	Decaps	KeyGen	Encaps	Decaps
Frost-128	878	586	668	401	569	666
Frost-192	2,428	1,421	1,596	999	1,367	1,551
Frost-256	5,153	2,929	3,139	2,010	2,638	2,873
Frost-384	11,387	7,406	7,892	3,915	6,213	6,650
Frost-CC-384	13,954	6,723	7,044	5,378	6,110	6,455
Frost-512	21,277	14,673	15,195	6,805	12,465	12,902
Frost-CC-512	26,904	12,028	12,555	10,582	10,283	10,789

The compact-ciphertext layout shifts matrix width from the ciphertext side to the public-key side. For the high-security compact-ciphertext instances, this increases the public-key-side work in key generation while reducing the online ciphertext-side matrix width in encapsulation and decapsulation. For this reason, **Frost-CC-384** and **Frost-CC-512** have larger key-generation costs but lower encapsulation and decapsulation costs than the corresponding default high-security instances.

References

- [ACD⁺18] Martin R Albrecht, Benjamin R Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the $\{\text{LWE}, \text{NTRU}\}$ schemes! In *International Conference on Security and Cryptography for Networks*, pages 351–367. Springer, 2018.
- [Alb17] Martin R Albrecht. On dual lattice attacks against small-secret lwe and parameter choices in helib and seal. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 103–129. Springer, 2017.
- [APS15] Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Cryptology ePrint Archive*, 2015.

- [BBF⁺19] Hayo Baan, Sauvik Bhattacharya, Scott R. Fluhrer, Óscar García-Morchón, Thijs Laarhoven, Ronald Rietman, Markku-Juhani O. Saarinen, Ludo Tolhuizen, and Zhenfei Zhang. Round5: Compact and fast post-quantum public-key encryption. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pages 83–102, Chongqing, China, May 8–10, 2019. Springer, Cham, Switzerland.
- [BCD⁺16] Joppe W. Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! Practical, quantum-secure key exchange from LWE. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1006–1018, Vienna, Austria, October 24–28, 2016. ACM Press.
- [BLL⁺22] Lei Bi, Xianhui Lu, Junjie Luo, Kunpeng Wang, and Zhenfei Zhang. Hybrid dual attack on lwe with arbitrary secrets. *Cybersecurity*, 5(1):15, 2022.
- [BLP⁺13] Zvika Brakerski, Adeline Langlois, Chris Peikert, Oded Regev, and Damien Stehlé. Classical hardness of learning with errors. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 575–584, Palo Alto, CA, USA, June 1–4, 2013. ACM Press.
- [BOR⁺21] Joppe W. Bos, Maximilian Ofner, Joost Renes, Tobias Schneider, and Christine van Vredendaal. The matrix reloaded: Multiplication strategies in FrodoKEM. In Mauro Conti, Marc Stevens, and Stephan Krenn, editors, *CANS 21: 20th International Conference on Cryptology and Network Security*, volume 13099 of *Lecture Notes in Computer Science*, pages 72–91, Vienna, Austria, December 13–15, 2021. Springer, Cham, Switzerland.
- [CKLS18] Jung Hee Cheon, Duhyeong Kim, Joohee Lee, and Yongsoo Song. Lizard: Cut off the tail! A practical post-quantum public-key encryption from LWE and LWR. In Dario Catalano and Roberto De Prisco, editors, *SCN 18: 11th International Conference on Security in Communication Networks*, volume 11035 of *Lecture Notes in Computer Science*, pages 160–177, Amalfi, Italy, September 5–7, 2018. Springer, Cham, Switzerland.
- [CN11] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 1–20, Seoul, South Korea, December 4–8, 2011. Springer Berlin Heidelberg, Germany.
- [EJK20] Thomas Espitau, Antoine Joux, and Natalia Kharchenko. On a dual/hybrid approach to small secret lwe: A dual/enumeration technique for learning with errors and application to security estimates of the schemes. In *International Conference on Cryptology in India*, pages 440–462. Springer, 2020.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO’99*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany.
- [GJ21] Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving lwe with applications to crystals. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 33–62. Springer, 2021.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer*

- Science*, pages 341–371, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.
- [HPRR20] James Howe, Thomas Prest, Thomas Ricosset, and Mélissa Rossi. Isochronous Gaussian sampling: from inception to implementation: With applications to the falcon signature scheme. In *Post-Quantum Cryptography: 11th International Conference, PQCrypto 2020, Paris, France, April 15–17, 2020, Proceedings 11*, pages 53–71. Springer, 2020.
 - [JZC⁺18] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 96–125, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
 - [JZM19] Haodong Jiang, Zhenfeng Zhang, and Zhi Ma. Tighter security proofs for generic key encapsulation mechanism in the quantum random oracle model. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pages 227–248, Chongqing, China, May 8–10, 2019. Springer, Cham, Switzerland.
 - [KRR⁺18] Angshuman Karmakar, Sujoy Sinha Roy, Oscar Reparaz, Frederik Vercauteren, and Ingrid Verbauwhede. Constant-time discrete Gaussian sampling. *IEEE Transactions on Computers*, 67(11):1561–1571, 2018.
 - [LLL24a] Shanxiang Lyu, Ling Liu, and Cong Ling. Learning with quantization: A ciphertext efficient lattice problem with tight security reduction from LWE. Cryptology ePrint Archive, Paper 2024/714, 2024.
 - [LLL⁺24b] Shanxiang Lyu, Ling Liu, Cong Ling, Junzuo Lai, and Hao Chen. Lattice codes for lattice-based pke. *Designs, Codes and Cryptography*, 92(4):917–939, 2024.
 - [MSC10] Robby G McKilliam, Warren D Smith, and I Vaughan L Clarkson. Linear-time nearest point algorithms for coxeter lattices. *IEEE transactions on information theory*, 56(3):1015–1022, 2010.
 - [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th Annual ACM Symposium on Theory of Computing*, pages 84–93, Baltimore, MA, USA, May 22–24, 2005. ACM Press.
 - [WZZ⁺24] Anyu Wang, Zhongxiang Zheng, Chunhuan Zhao, Zhiyuan Qiu, Guang Zeng, Ye Yuan, Changchun Mu, and Xiaoyun Wang. Scloud+: An efficient lwe-based kem without ring/module structure. In *International Conference on Research in Security Standardisation*, pages 147–174. Springer, 2024.
 - [Zam14] Ram Zamir. *Lattice coding for signals and networks: A structured coding approach to quantization, modulation, and multiuser information theory*. Cambridge University Press, 2014.